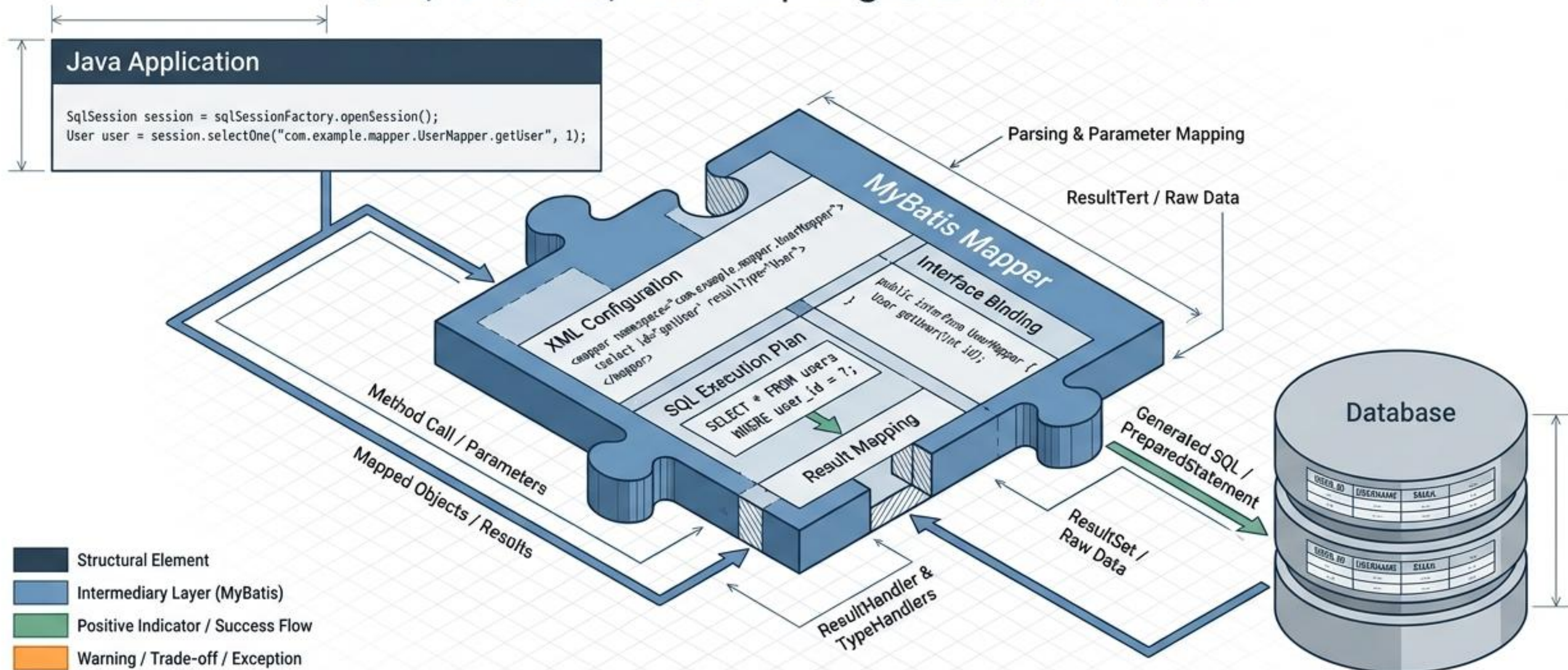


# MyBatis: 정밀한 SQL 제어의 미학

구조, 동작 원리, 그리고 Spring 통합까지 완벽 가이드





# 한눈에 보는 MyBatis: 핵심 아키텍처와 동작 원리

## MyBatis의 핵심 가치와 구성 요소



## 3대 핵심 구성 파일



## Spring + MyBatis 처리 흐름



## 주요 클래스의 역할과 생명주기

| 구성 요소             | 역할                  | 생명주기 (Scope)                     |
|-------------------|---------------------|----------------------------------|
| SqlSessionFactory | SqlSession을 생성하는 공장 | 어플리케이션 실행 동안 유지                  |
| SqlSession        | SQL 실행 및 트랜잭션 관리    | 요청 또는 메서드 단위 (Thread-safe 하지 않음) |
| Mapper Interface  | SQL 호출을 위한 자바 인터페이스 | SqlSession과 동일 (명시적 해제 불필요)      |



# MyBatis 핵심 가이드: 동적 쿼리 & 세션 관리 마스터

## 실무 핵심 동적 SQL 태그

```
<sql>
MyBatis ...

<where>
  <set>
    connection
  </set>
</where>

<foreach>
  SET name =
    'New',
    age = 30
</foreach>
...
```



**<where> & <set>의 자동 최적화**

SQL  
WHERE state = 'ACTIVE'

조건에 따른 WHERE 절 생성

SQL  
SET name = 'New', age = 30

UPDATE 문에서의 불필요한  
coma(,) 자동 제거



**<foreach>를 활용한  
IN 조건 처리**

[ 1,  
2,  
3]

SQL  
IN (1, 2, 3)



## #{ } 사용률 99% 권장

SQL Injection 방지를 위해 보안에 안전한 #{ } 방식 사용



## 동적 SQL 태그 요약

| 태그        | 진문               | 차별                                  |
|-----------|------------------|-------------------------------------|
| <if>      | Condition Search | 특정 조건 만족 시에만 SQL 조각 포함              |
| <where>   | WHERE Generation | 하위 조건이 없을 시 WHERE 생략 및 AND/OR 자동 제거 |
| <foreach> | Iteration        | List, Array 파라미터를 IN 절로 변환          |

## MyBatis 핵심 체제 및 동작 구조



### SqlSessionFactory

설정 정보 보관 '공장'  
애플리케이션 시작 시 1개 생성  
안전함 (O)



### SqlSession

실제 DB 작업 '작업 공간'  
요청 시마다 생성  
안전하지 않음 (X)

## JDBC vs MyBatis 실행 흐름

### JDBC



DriverManager



Connection



Statement



ResultSet

### MyBatis



SqlSessionFactory



SqlSession



Mapper Interface



Result

Factory-Session-Mapper 구조로 추상화

## Spring Boot의 자동화



SqlSessionFactory

| 항목          | SqlSessionFactory  | SqlSession       |
|-------------|--------------------|------------------|
| 생성 시점       | 애플리케이션 서약 시 1개     | 오징 시마다 생성        |
| Thread Safe | 안전함 (D)            | 안전하지 않음 (X)      |
| 주요 역할       | 설정 로딩 및 Session 생성 | SQL 실행 및 프렌잭션 엔티 |

실무에서는 Spring이 자동 처리하여 때퍼 인터페이스만 주입받아 사용



# 자바 백엔드 개발의 핵심 4요소: 어노테이션부터 MVC 패턴까지

## 설정과 데이터의 구조화 (Annotation & XML)

```
1 @Controller
2 public class het {
3     public roconious() {
4     }
5 }
6
7
8
9 }
```

@Controller @Service @Autowired

### 어노테이션: 코드 위의 '스마트 포스트잇'

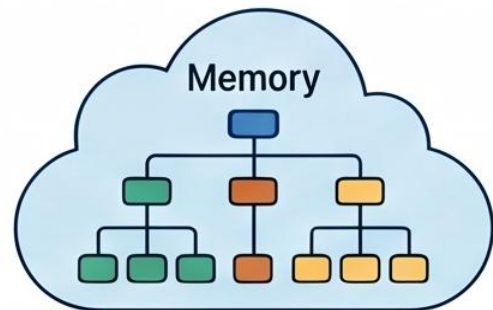
컴파일러나 프로그램이 읽고 특정 작업을 수행하도록 지시하는 직관적인 설정 방식입니다.

```
<bean>
  <property>
    <value>
  </property>
</bean>
```

### XML: 데이터를 담는 '맞춤형 상자'

태그를 사용하여 데이터의 의미를 정의하고 구조화하여 전달하는 텍스트 기반 형식입니다.

## DOM vs SAX 파싱 방식



DOM (메모리 전체 로드)  
전체를 메모리에 올려 자유롭게 가공



SAX (순차적 읽기)  
한 줄씩 가볍게 읽는 방식

## 실행 로직과 시스템 설계 (MyBatis & MVC)

```
1 Java
2 public class {
3     public List<User>
4     selectAll() {
5     }
6     ...
7 }
8 }
```

MyBatis Mapper

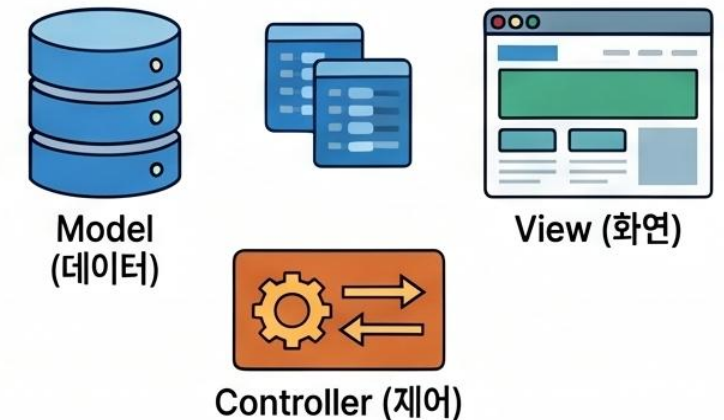
### MyBatis: 강력한 동적 SQL 매퍼

SQL을 자바 코드와 분리하고, 상황에 따라 쿼리가 변하는 동적 기능을 제공합니다.



### MVC 패턴: 역할 분리의 정석

Model(데이터), View(화면), Controller(제어)로 역할을 나누어 유지보수성을 극대화합니다.



## Forward vs Redirect의 차이

### Forward (포워드)

URL 주소장: 변하지 않음 (기존 유지)

Request 객체: 그대로 유지됨

주요 용도:  
조회(SELECT) 기능 수행 시

### Redirect (커다이펙트)

URL 주소장: 변환 (새 주소로 변경)

Request 객체: 소멸 후 새 객체 생성

주요 용도:  
등록/수정/삭제(CUD) 수행 후



## Low Level: JDBC

직접 제어, 높은 복잡도

커넥션 관리,  
PreparedStatement  
생성, ResultSet 처리 등  
반복적이고 번거로운  
보일러플레이트 코드 발생.

## Mid Level: SQL Mapper (MyBatis)

SQL 중심, 유연성 극대화

JDBC의 복잡한 설정을  
**간소화하고 자동화**. 개발자가  
직접 SQL을 작성하여 자바  
객체(VO, DTO)에 매핑.

## High Level: ORM (JPA, Hibernate)

객체 중심, 높은 추상화

객체를 통해 간접적으로 DB를  
다루며 SQL을 자동 생성.

추상화 수준 (Level of Abstraction)

# 아키텍처 의사결정: SQL Mapper vs. ORM

|         | MyBatis (SQL Mapper)                                                                                                        | JPA (ORM)                                                                                                          |
|---------|-----------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| 접근 방식   | 직접 작성한 SQL과 객체를 매핑                                                                                                          | 객체 그래프와 관계를 DB 테이블에 매핑                                                                                             |
| 핵심 장점   | 복잡한 조인, 서브쿼리 작성 및 정교한 DBA 수준의 SQL 튜닝 용이  | 반복적인 기본적인 CRUD 자동화로 개발 생산성 극대화  |
| 핵심 단점   | DB 스키마 종속성 심화, 기본 CRUD 쿼리 수동 작성의 번거로움  | 복잡한 통계 쿼리 작성의 한계, 높은 러닝 커브    |
| 최적의 사용처 | 금융권, 공공기관, 레거시 DB 연동, 대용량 트래픽 통계 서비스                                                                                        | 신규 프로젝트, 도메인 주도 설계(DDD) 기반 서비스                                                                                     |



# MyBatis 도입의 명암 (Pros & Cons)

## 강점 (Pros)



### 정밀한 직접 제어

특정 DB에 최적화된 SQL을 개발자가 100% 제어.



### 코드 분리

자바 프로그램 코드와 SQL 쿼리를 분리하여 유지보수성 향상.



### 결과 매핑의 유연성

VO 없이도 사용자 정의 DTO, MAP 등으로 자유로운 매핑 지원.



### 낮은 러닝 커브

직관적인 XML/어노테이션 기반 사용법.

## 약점 (Cons)



### 유지보수 비용

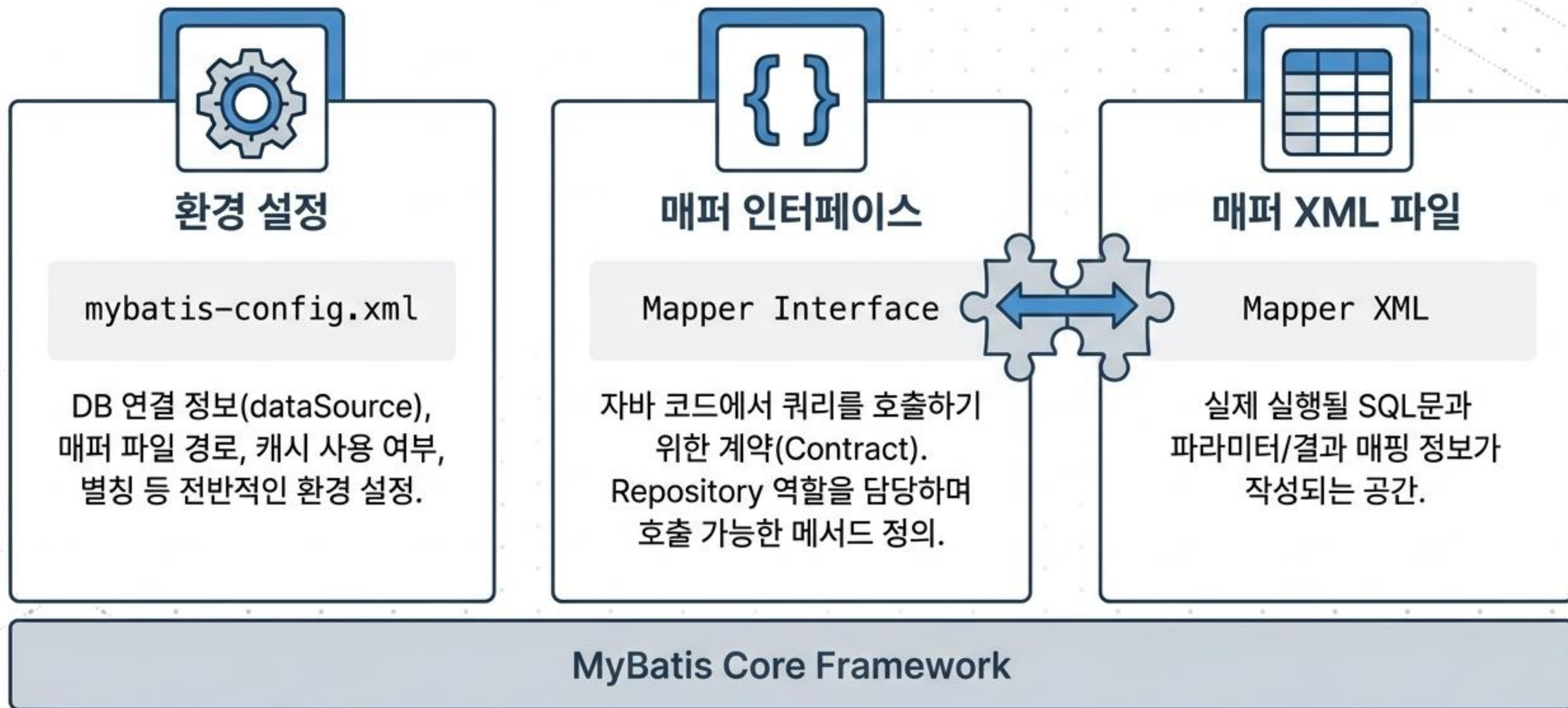
DB 변경 시 스키마에 종속적인 쿼리들을 모두 수정해야 함.



### 반복 작업

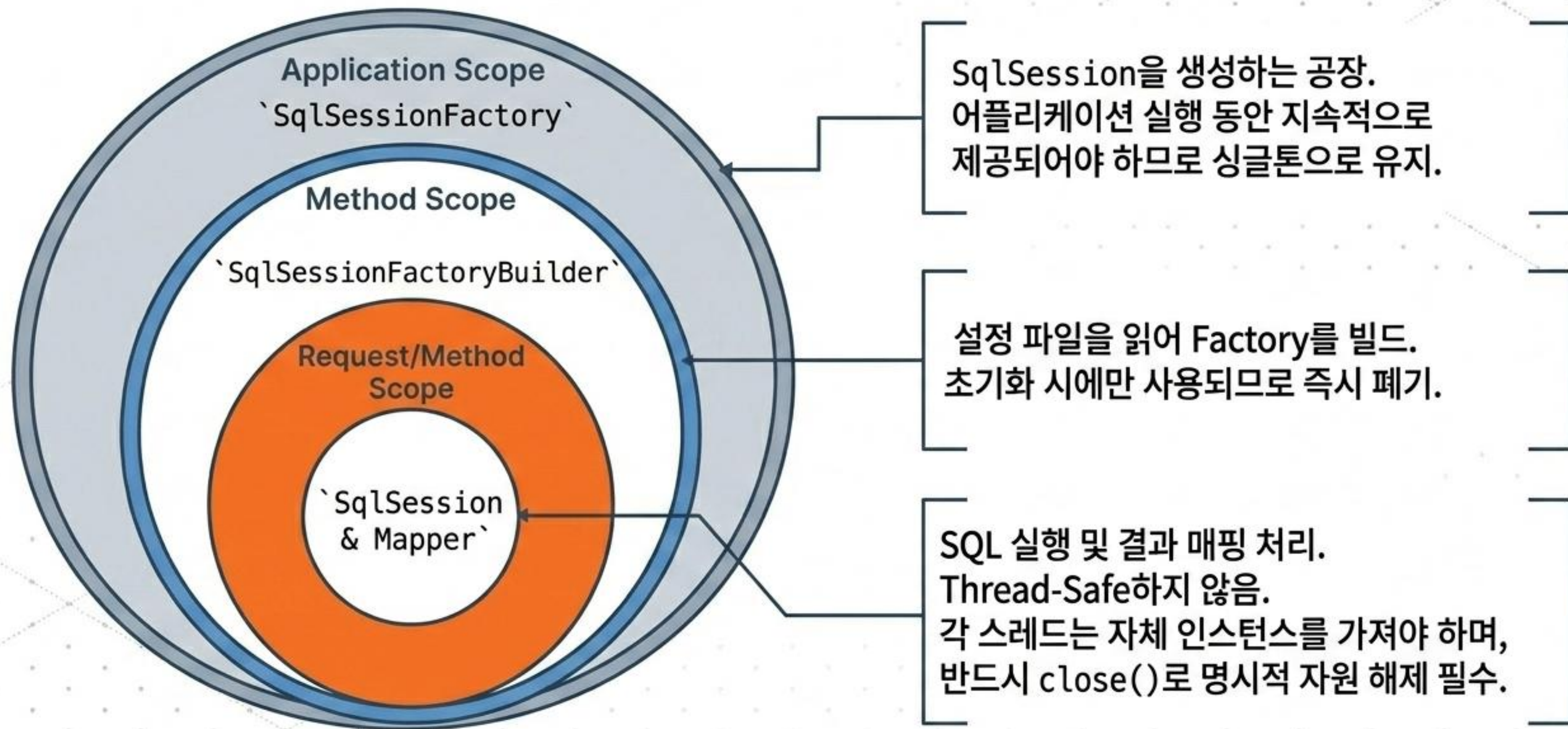
단순하고 비슷한 기본 CRUD 쿼리를 남발하게 되는 경향.

# 시스템의 뼈대: 3대 핵심 구성 요소



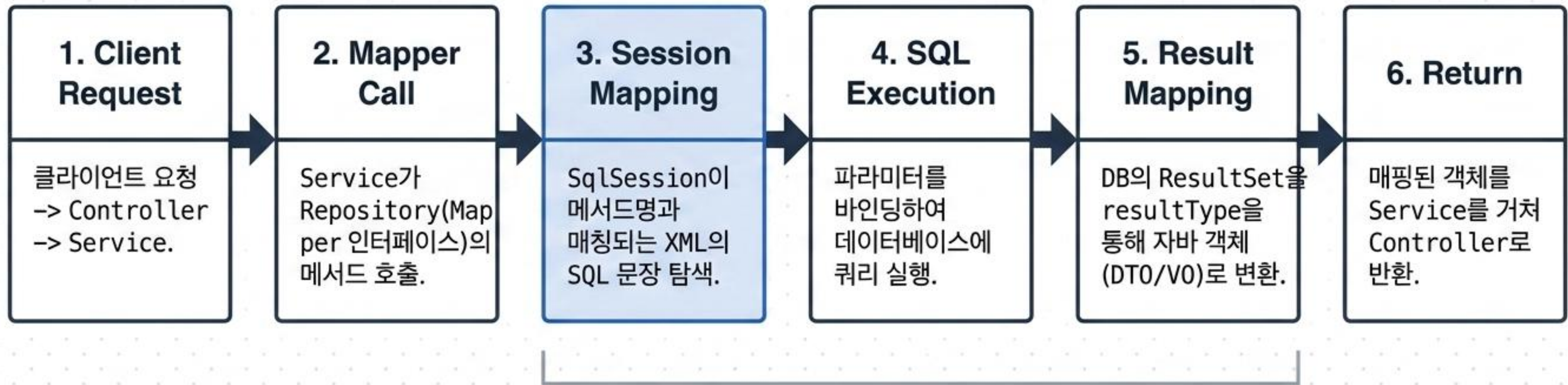


# 객체 생명주기와 스코프 관리 전략





# 데이터의 여정: 쿼리 실행 파이프라인



MyBatis Core Processing



# SQL 매핑의 두 가지 방식: Annotation vs. XML

## Annotation 방식 (Java 내부)

```
@Select("SELECT * FROM computer WHERE  
name = #{name}")  
List<Computer> findByName(@Param("name")  
String name);
```

### Tag: 빠르고 간결함

XML 파일 없이 인터페이스 내에 직접 작성.  
단순한 CRUD 쿼리에 적합.

VS

## XML 방식 (외부 파일)

```
<select id="findByName"  
parameterType="String"  
resultType="Computer">  
    SELECT * FROM computer WHERE  
    name = #{name}  
</select>
```

### Tag: 복잡성 제어

SQL 쿼리를 별도 파일로 분리. 복잡한 조인이나  
동적 SQL 작성 시 압도적으로 유리.



# 매퍼 XML의 해부학 (Anatomy of Mapper XML)

## 식별자와 반환타입

**id**: SQL 매핑을 위한 고유 구분자. (인터페이스 메서드명과 일치)  
**resultType**: 단일 결과값의 데이터 타입.

```
<select id="findUser" resultType="UserMap">
  SELECT * FROM users
  WHERE id = #{paramId} AND status = '${subStatus}'
</select>
```

## 파라미터 바인딩의 차이

**#{param}**: PreparedStatement를 사용하여 파라미터를 안전하게 바인딩 (SQL Injection 방지).

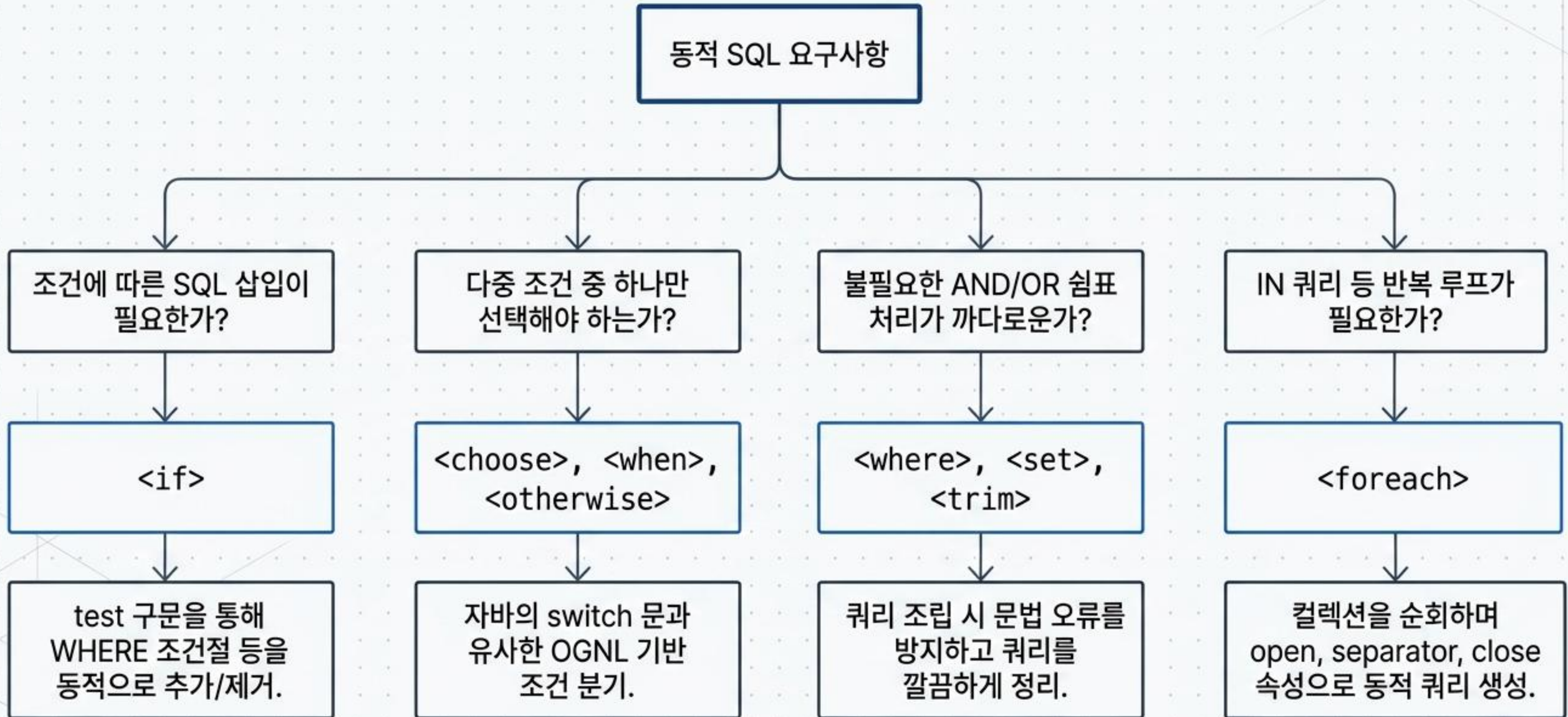
**\${subb}**: SQL 문자열 자체를 직접 치환 (동적 테이블명 등에 사용).

## resultMap (복잡한 매핑)

- DB 컬럼과 객체 프로퍼티 불일치 해결.
- **association**: Has-one 관계
- **collection**: Has-many 관계



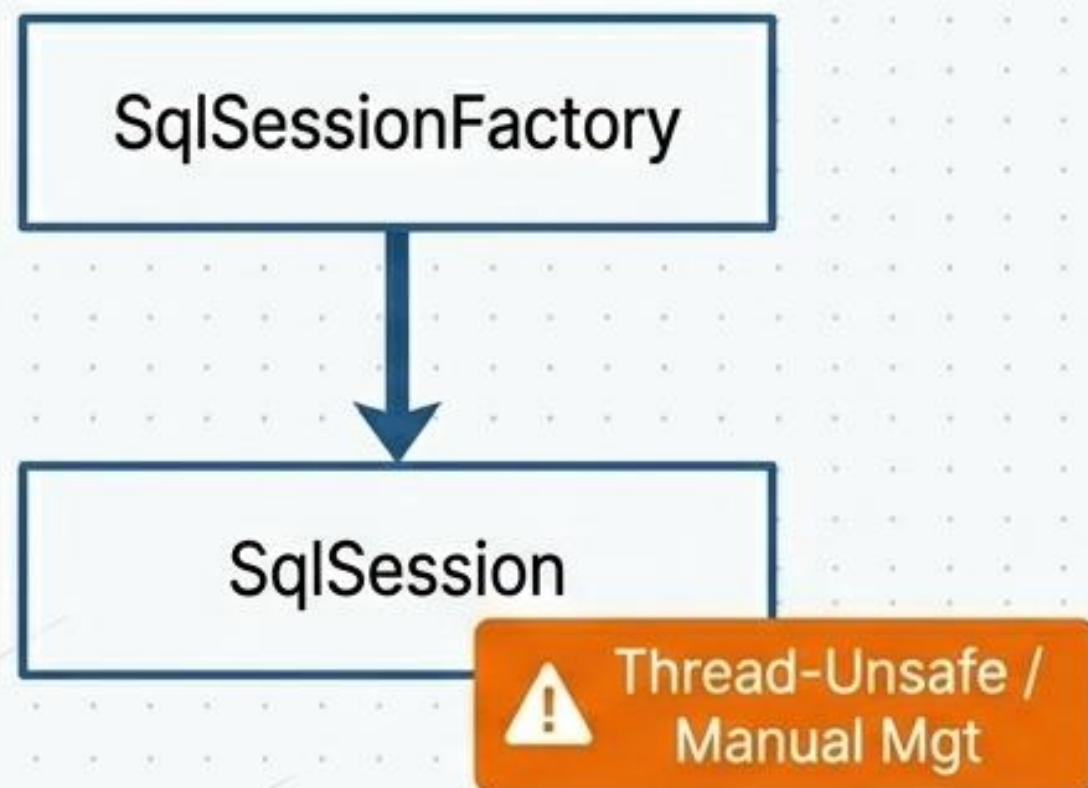
# MyBatis의 강력한 무기: 동적 SQL (Dynamic SQL)





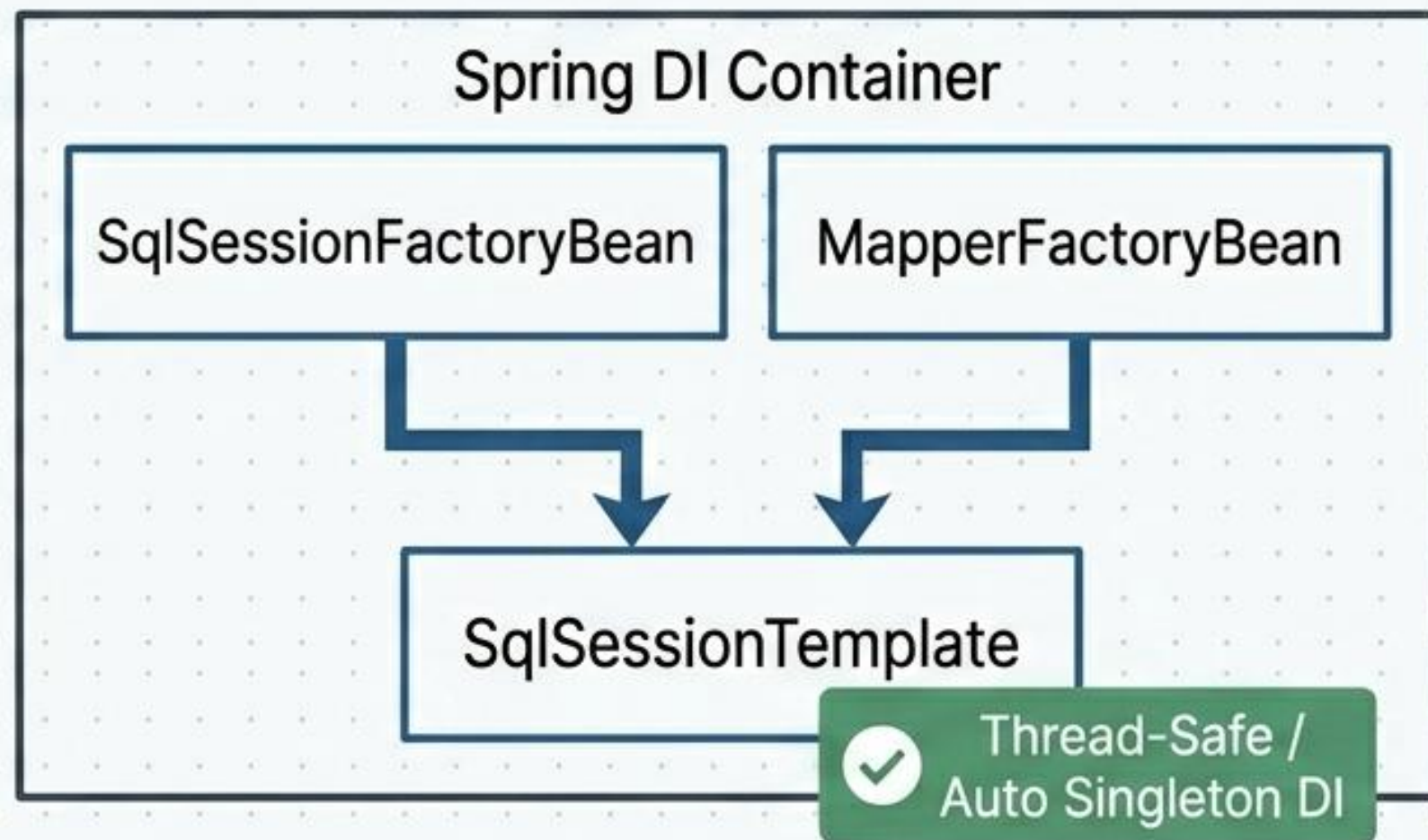
# 아키텍처의 진화: 표준 MyBatis vs. MyBatis-Spring

## Before (표준 MyBatis3)



개발자가 직접 인스턴스를 스레드별로 할당하고 close()를 호출해야 함.

## After (MyBatis-Spring 도입)



SqlSessionTemplate이 안전한 싱글톤 프록시 객체를 제공. 스프링의 트랜잭션 관리와 완벽하게 연동되어 수동 자원 해제가 불필요해짐.



# 완벽한 자동화: MyBatis-Spring 동작 시퀀스



1

## Application Startup

MapperFactoryBean이 스레드-세이프한 프록시 매퍼 객체와 SqlSessionTemplate을 생성해 Spring DI 컨테이너에 싱글톤으로 등록.

2

## Client Request

Service가 DI 주입받은 프록시 매퍼 메서드 호출.

3

## Template Delegation

프록시가 SqlSessionTemplate으로 위임.

4

## Transaction Sync

스프링 트랜잭션에 동기화된 표준 SqlSession을 획득.

5

## SQL Execution

트랜잭션 내에서 동일한 SqlSession을 재사용하여 DB 쿼리 실행 후 자동 반환.





# 실전 구현: Spring Boot 기반 CRUD 매핑

## 1. The Schema

| computer 테이블 |      |       |
|--------------|------|-------|
| id           | name | price |

(JPA와 달리 @Entity 불필요)

## 2. The Mapper (ComputerMapper.java)

```
@Mapper 명시
@Select("SELECT * FROM computer")
List<Computer> findAll();

@Update("INSERT INTO computer (name,
price) values (#{name}, #{price})")
void saveComputerInfo(...);
```

## 3. The Controller (ComputerController.java)

생성자 의존성 주입 (DI)

```
computerMapper.findByName("늑대와여우");
-> 비즈니스 로직으로 즉시 활용.
```



# 설계자의 결론: 언제 MyBatis를 선택해야 하는가?

## 적극 권장 (Perfect Fit)

- ✓ 금융권, 공공기관 등 복잡한 조인과 다중 서브쿼리가 비즈니스 로직의 핵심일 때.
- ✓ DBA의 엄격한 쿼리 튜닝 요구사항을 100% 수용해야 할 때.
- ✓ 복잡한 뷰나 프로시저가 얹혀있는 기존 레거시 데이터베이스와 연동할 때.
- ✓ JSTL/OGNL 등 동적 쿼리 작성이 빈번하게 일어나는 시스템.

## 재고 필요 (Reconsider)

- ✗ 테이블 스키마 변경이 잦은 초기 스타트업 단계의 신규 프로젝트.
- ✗ 단순 CRUD 위주의 도메인 주도 설계(DDD)가 필요한 서비스 (이 경우 JPA/ORM 권장).



# 핵심 요약 (Key Takeaways)

## 1. 정밀한 제어력 (Precision Control)

JDBC의 보일러플레이트를 제거하면서도, 개발자에게 100%의 SQL 제어권과 튜닝 권한을 부여하는 최적의 타협점입니다.

## 2. 동적 쿼리의 강력함 (Dynamic Power)

XML 기반의 `<if>`, `<choose>`, `<foreach>` 등의 태그를 통해 어떤 복잡한 비즈니스 조건의 쿼리도 유연하게 조립할 수 있습니다.

## 3. Spring과의 완벽한 통합 (Seamless Integration)

SqlSessionTemplate과 프록시 객체를 통한 DI 및 트랜잭션 자동화로, 스레드-세이프티 걱정 없이 안정적인 엔터프라이즈 환경을 구축합니다.



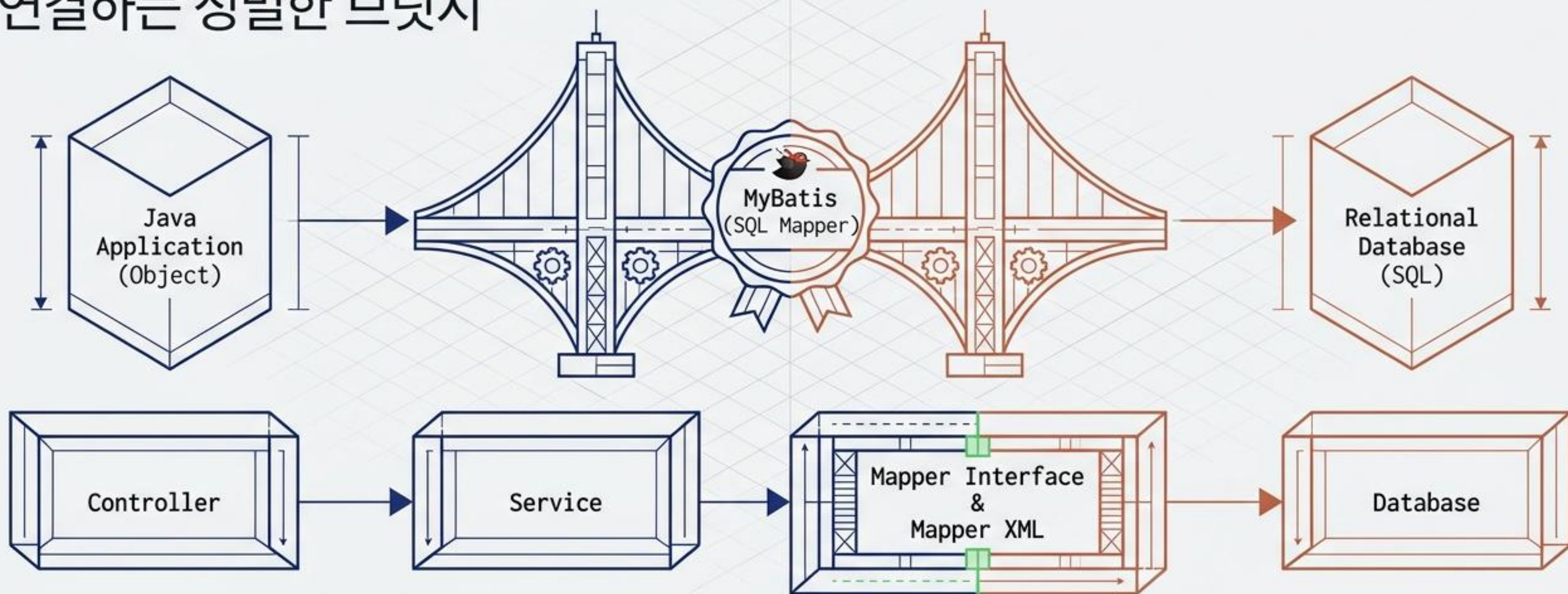


# MyBatis Masterclass: 핵심 개념부터 동적 쿼리, 내부 엔진까지

SQL 제어의 정밀함과 동적 쿼리의 유연성을 완벽하게 다루는 실무 가이드



# 객체와 관계형 데이터베이스를 연결하는 정밀한 브릿지



## 핵심 가치

JPA처럼 SQL을 자동 생성하지 않고,  
개발자가 SQL을 직접 제어하는 프레임워크.

## 강점

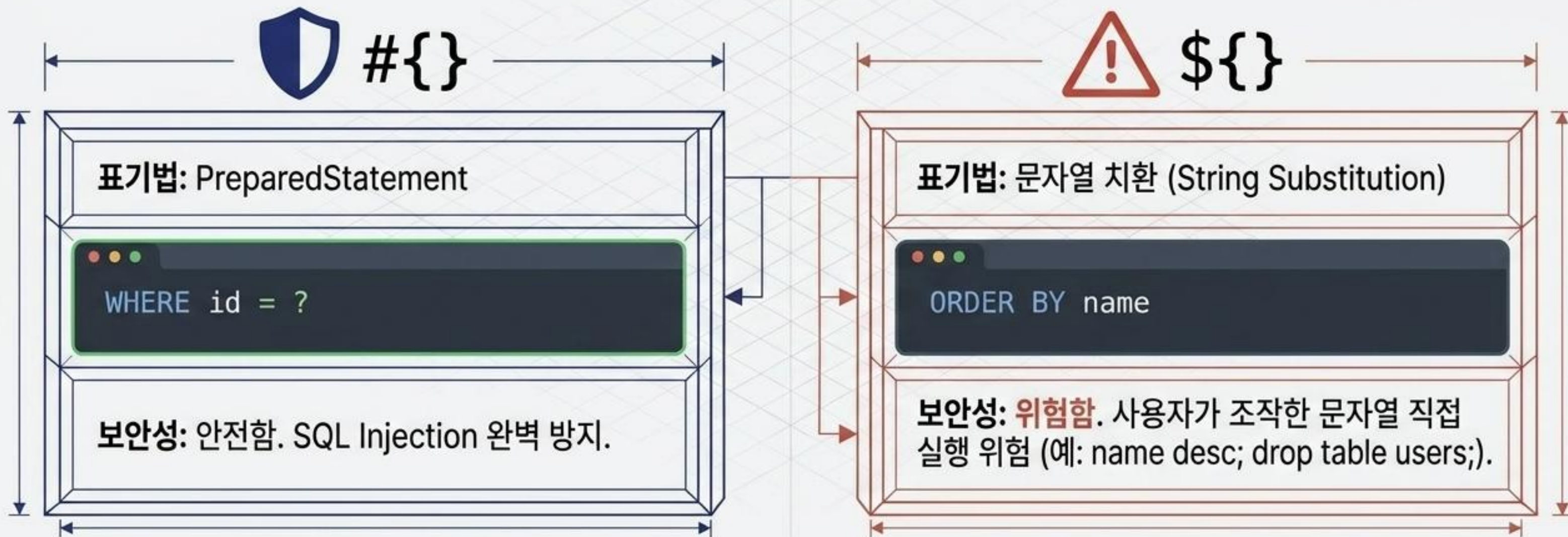
복잡한 쿼리 작성의 용이성, 세밀한 성능 튜닝,  
JDBC 보일러플레이트 코드 자동화.

## 작동 방식

Java 메서드(Mapper Interface)와  
SQL 쿼리(Mapper XML)를 1:1로 매핑.



# 데이터 입력과 보안: 안전한 파라미터 바인딩 원칙



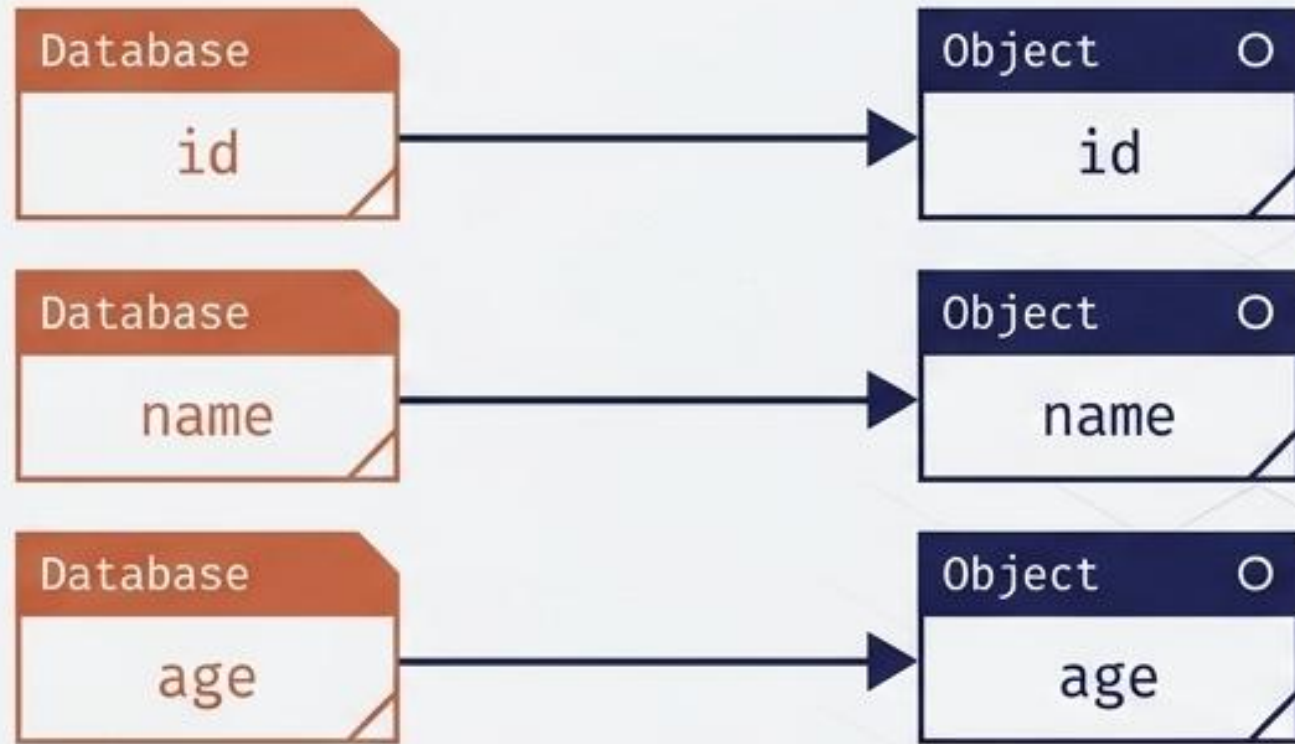
실무 원칙: 99% `#{}`  사용. 동적 정렬(ORDER BY) 등 테이블/컬럼명이 동적으로 바뀌어야 하는 극히 예외적인 상황에서만 `${}`  주의해서 사용.



# 데이터 출력: 컬럼과 객체 필드의 매핑 전략

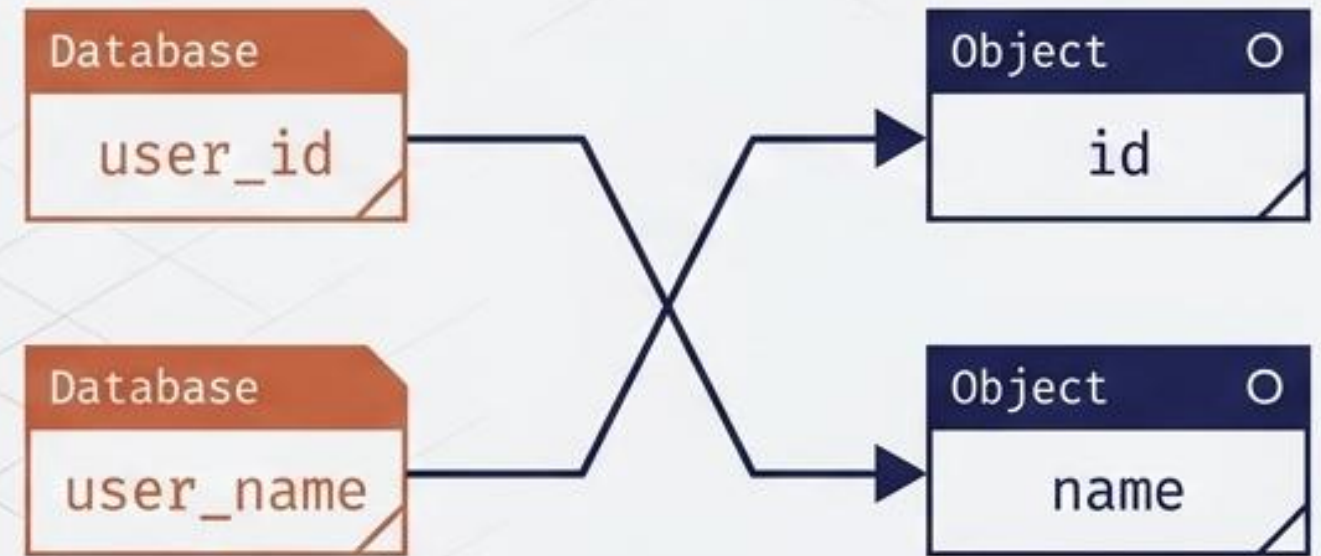


## resultType (자동 매핑)



DB 컬럼명과 Java DTO 필드명이 정확히 일치할 때 프레임워크가 자동으로 매핑.

## resultMap (수동/복잡한 매핑)



컬럼명과 필드명이 다르거나, 객체 내부에 또 다른 객체가 포함된 복잡한 구조일 때 **XML**에 명시적으로 매핑 규칙을 작성.



# 동적 쿼리의 시작과 한계 : 기본 if 태그의 문제점

동적 쿼리란?

조건에 따라 SQL 문장이 동적으로  
변하는 MyBatis의 핵심 기능.

```
<if test='name != null'>  
  AND name = #{name}  
</if>  
<if test='age != null'>  
  AND age = #{age}  
</if>
```

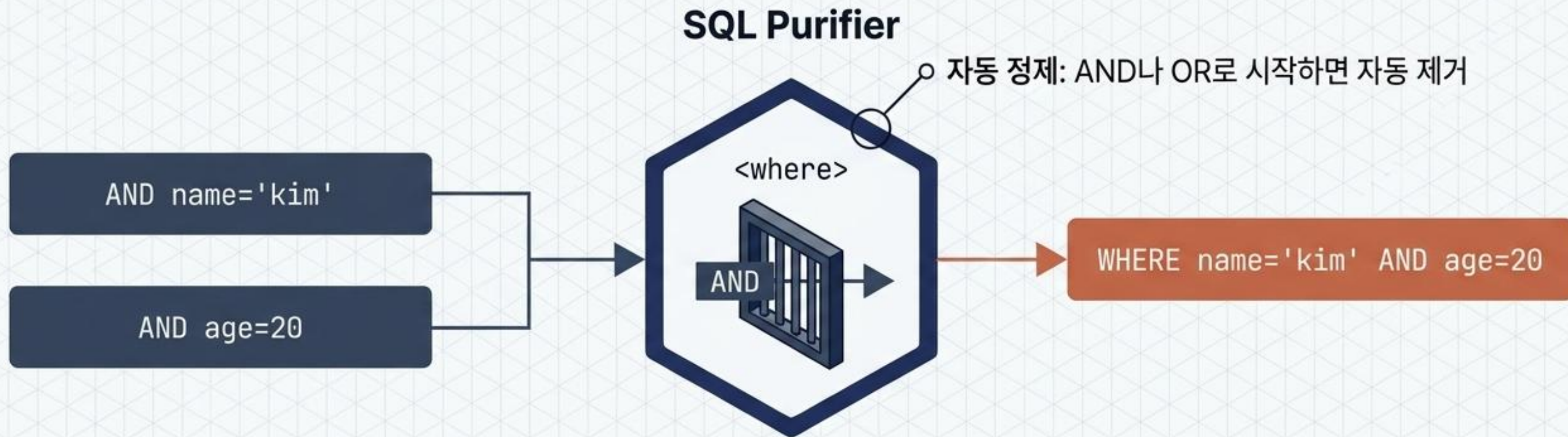
첫 번째 조건 누락,  
두 번째 조건만 참일 때

```
$ SELECT * FROM users AND age=20
```

문제 발생: 문법 오류 (Dangling Operator).  
단순 <if>의 한계를 극복할 스마트  
태그가 필요함.



# 스마트 필터링: 문맥을 이해하는 Where 태그



- **스마트 동작 원리:**

내부에 포함된 조건이 하나라도 만족하면 자동으로 WHERE 절을 생성.

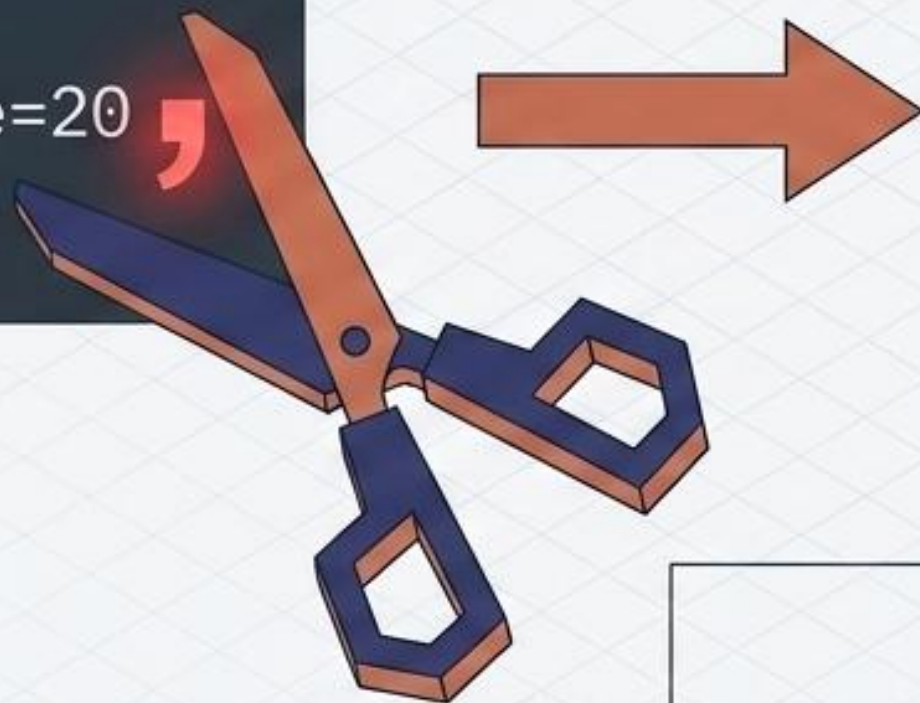
- **빈 조건 처리:**

모든 조건이 null이면 WHERE 절 자체를 아예 생성하지 않아 `SELECT * FROM users` 로 깔끔하게 실행.



# 스마트 업데이트: 쉼표를 정리하는 Set 태그

```
UPDATE users  
SET name='kim', age=20  
WHERE id=1
```

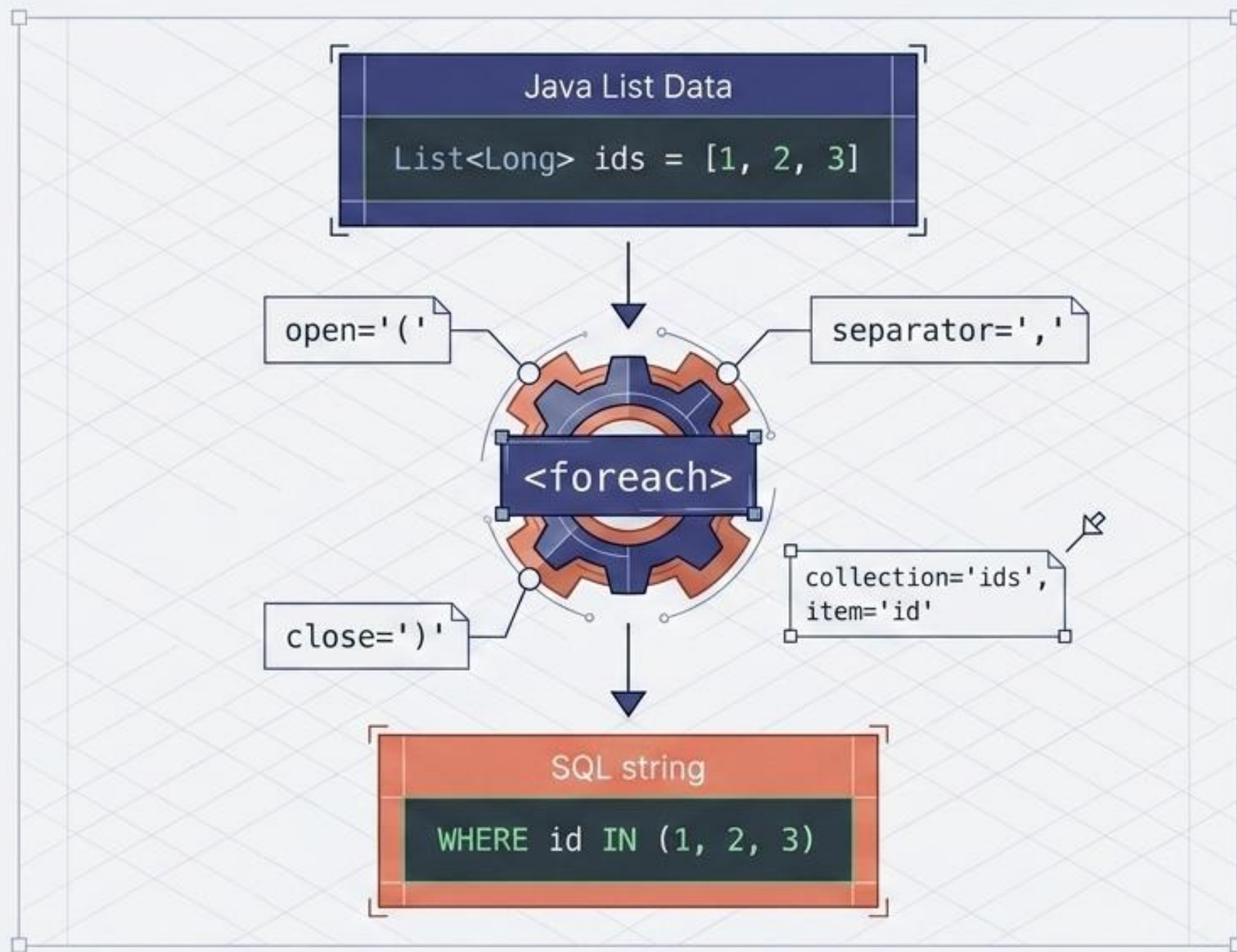


```
UPDATE users  
SET name='kim', age=20  
WHERE id=1
```

- 1 동적 부분 업데이트: 전달된 데이터(null이 아닌 값)만 선택적으로 업데이트할 때 필수.
- 2 자동 정제: 마지막 업데이트 항목 뒤에 남는 불필요한 쉼표(,)를 엔진이 감지하고 자동으로 잘라냄.



# 컬렉션 처리: IN 조건 쿼리의 완벽한 자동화

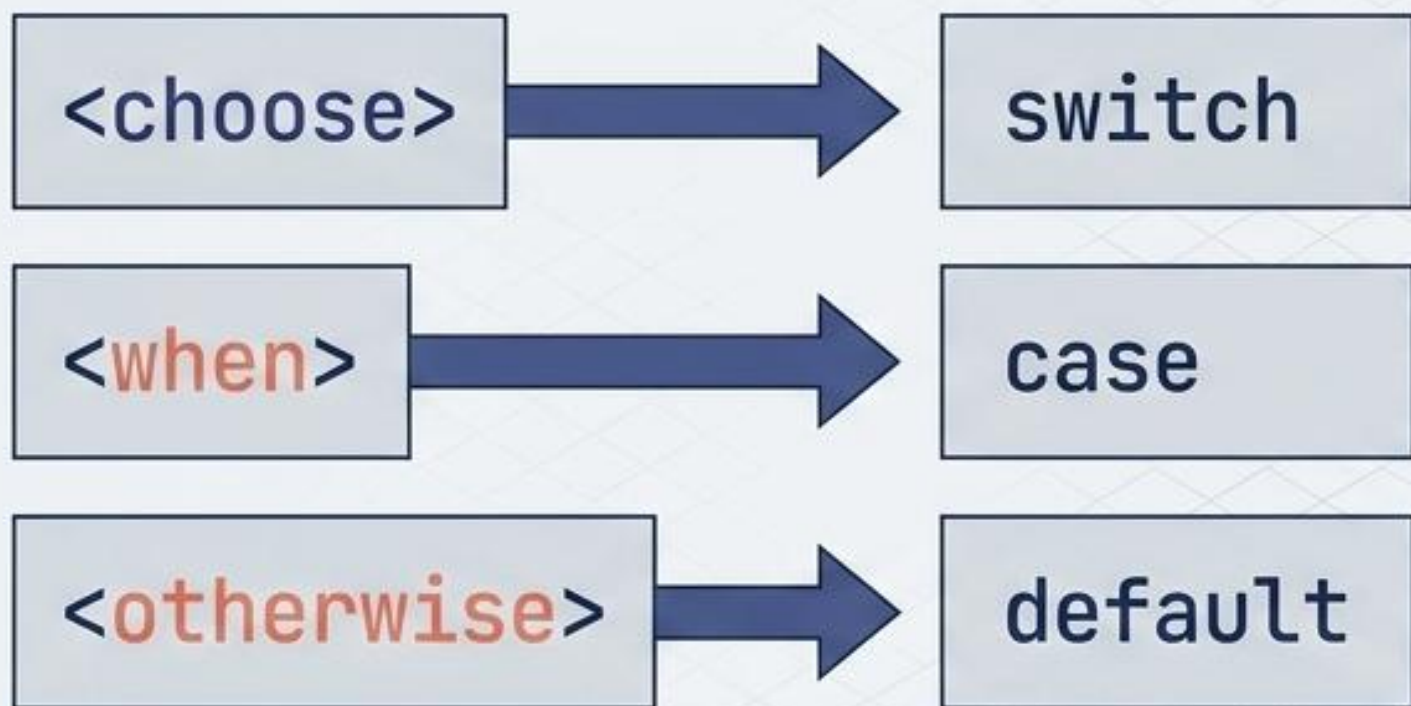


실무에서 매우 자주 사용하는 IN 조건의 동적 생성을 괄호와 심표 조립을 통해 완벽하게 기계화.



# 고급 제어 로직: Switch 분기와 커스텀 필터링

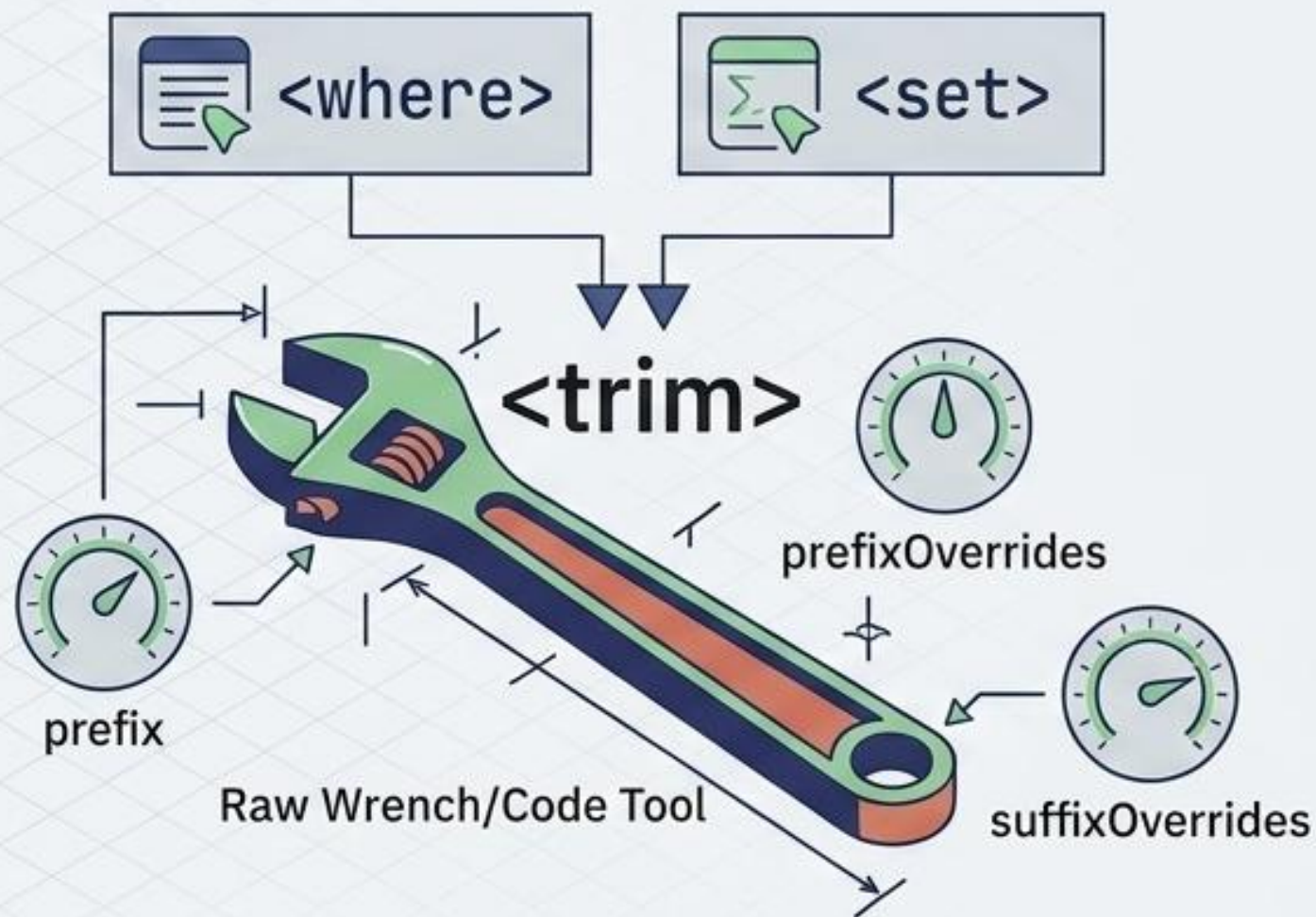
## Choose



## Choose

여러 조건 중 첫 번째로 참인 단 하나만 실행되는 상호 배타적 분기 로직.

## Trim

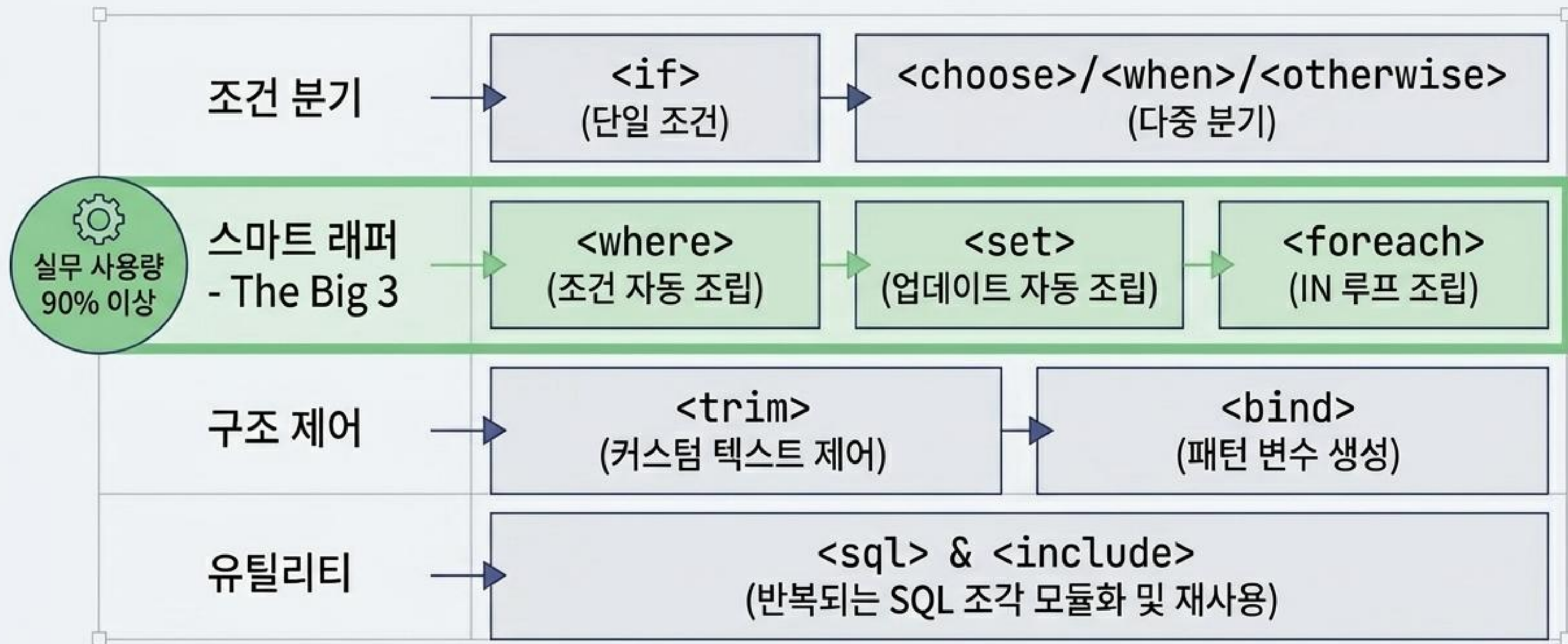


## Trim

`<where>`와 `<set>` 태그의 실제 원형. 직접 접두사와 제거할 단어를 커스텀하게 정의할 수 있는 로우레벨 도구.



# 동적 쿼리 마스터보드: 핵심 태그 치트시트



MySQL 페이징 팁: `LIMIT #{offset}, #{size}` 파라미터 바인딩과 결합하여 완벽한 동적 목록 조회 구현.



# 엔진 아키텍처: JDBC의 복잡성을 감추는 추상화

## Raw JDBC



## MyBatis Engine



MyBatis는 단순한 문자열 파서가 아닙니다. 내부적으로 커넥션 획득, 파라미터 매핑, 쿼리 실행, 예외 처리, 결과 객체 바인딩이라는 JDBC의 번거로운 파이프라인을 SqlSession이라는 단일 인터페이스 뒤로 완벽하게 숨깁니다.

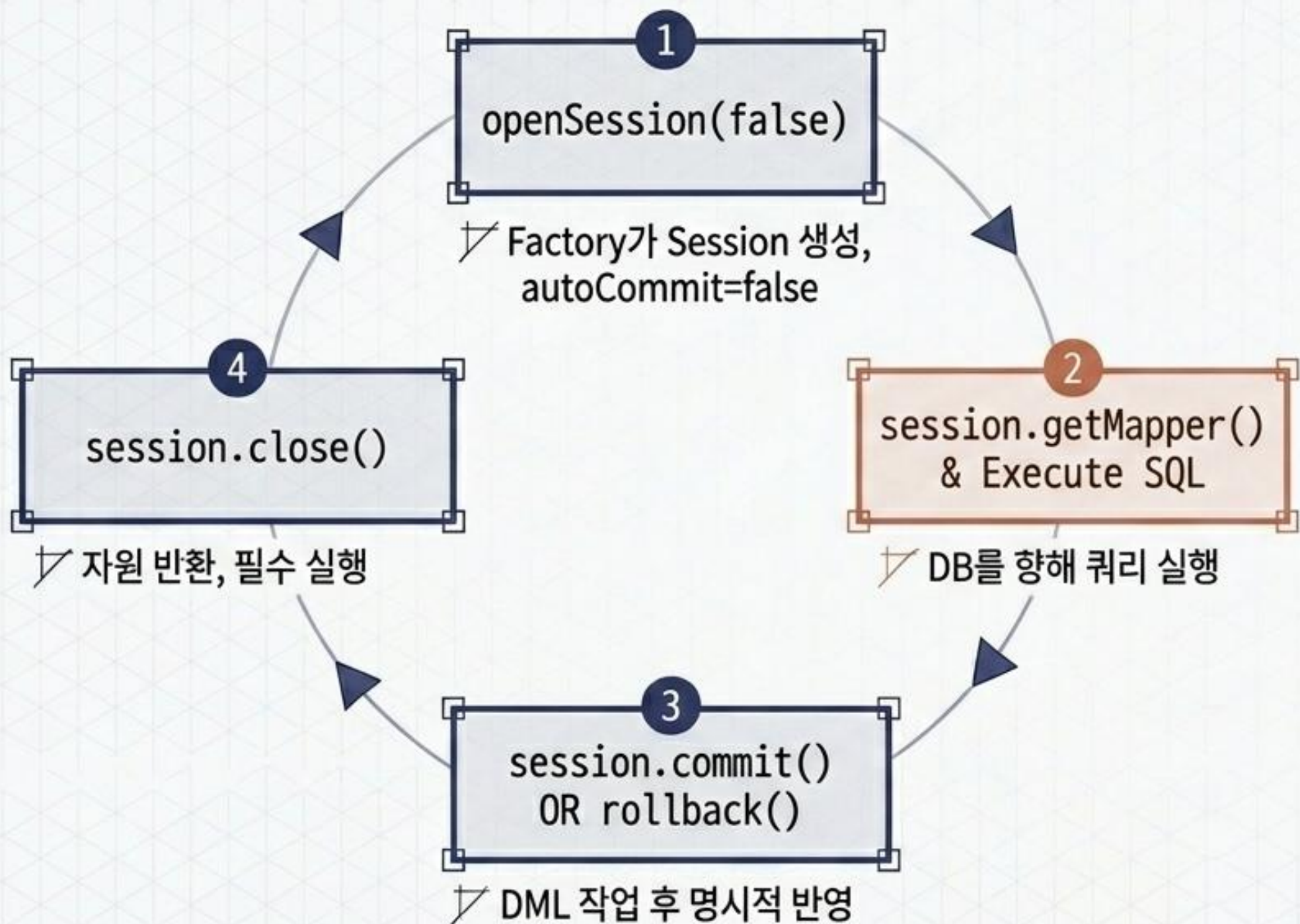


# 공장과 작업자: 핵심 엔진 객체의 역할과 수명

|                          | <div> SqlSessionFactory</div> | <div> SqlSession</div> |
|--------------------------|-----------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| 역할                       | ✓ 설정 정보 로딩 및 DB 연결 관리                                                                                           | ✓ 실제 SQL 실행 및 트랜잭션(Commit/Rollback) 관리                                                                    |
| 생성 횟수/수명                 | 애플리케이션 라이프사이클 전체<br>(1개만 생성, 싱글톤)                                                                               | HTTP 요청마다 생성되고 즉시 소멸                                                                                      |
| 스레드 안전성<br>(Thread Safe) | ✓ 안전함<br>(여러 스레드 공유 가능)                                                                                         | ✗ 안전하지 않음<br>(스레드 간 공유 절대 금지)                                                                             |



# 쿼리의 생명주기: 엄격한 세션과 트랜잭션 관리



## 핵심 규칙

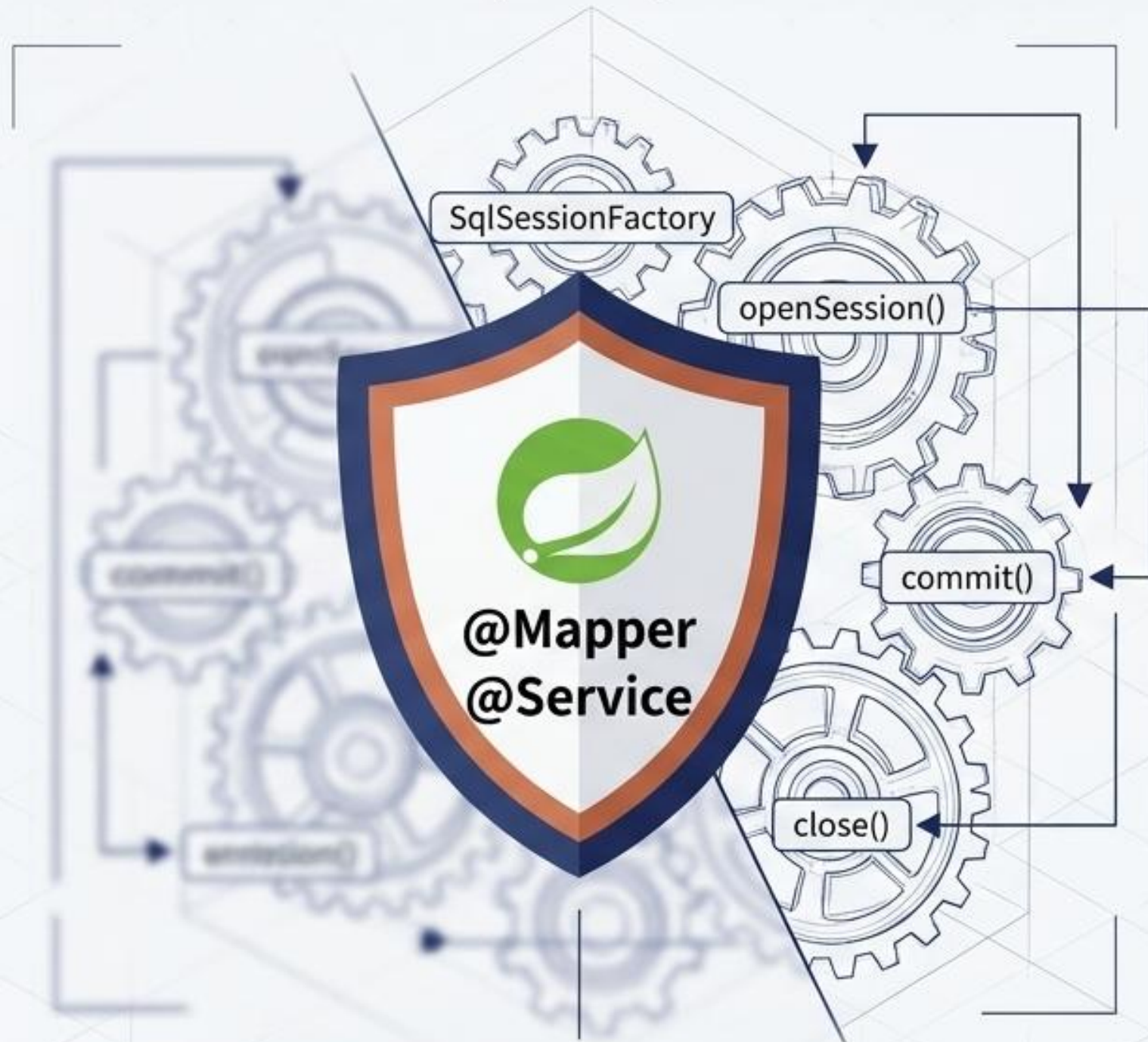
기본적으로 `openSession()`은 `autoCommit = false` 상태입니다.

따라서 DML(Insert, Update, Delete) 작업 후에는 반드시 명시적인 `commit()`이 호출되어야 실제 DB에 반영됩니다.

작업이 끝난 `SqlSession`은 커넥션 풀 반환을 위해 반드시 `close()` 해야 합니다.



# 모던 생태계: Spring Boot 환경의 완벽한 자동화



Spring Boot의 완벽한 자동화 커튼

- ✓ **현실의 실무:** 현대의 Spring Boot 환경에서는 개발자가 `SessionFactory`나 `Session`을 직접 다루지 않습니다.
- ✓ **스프링의 마법:** Spring의 MyBatis-Spring 모듈이 팩토리 생성, 세션 할당, 트랜잭션(`@Transactional`) 관리, 프록시 객체 생성을 전부 자동화합니다.
- ✓ **개발자의 역할:** 이제 여러분은 오직 데이터 파이프라인(DTO)과 설계도(XML 동적 쿼리) 작성에만 온전히 집중하면 됩니다.



# 실무 완벽 적용: 동적 검색 마스터 패턴

```
List<User> search(UserSearch cond);

<select id="search" resultType="User">
  SELECT * FROM users
  <where>
    <if test="name != null and name != ''">
      AND name LIKE CONCAT('%', #{name}, '%')
    </if>
    <if test="age != null">
      AND age = #{age}
    </if>
    <if test="city != null">
      AND city = #{city}
    </if>
  </where>
</select>
```

**보안:** 모든 파라미터는 #{ }로 바인딩되어 SQL 인젝션 원천 차단.

**동적 조립:** 이름, 나이, 도시 조건의 조합에 따라 WHERE 절과 AND가 완벽한 문법으로 자동 정제.

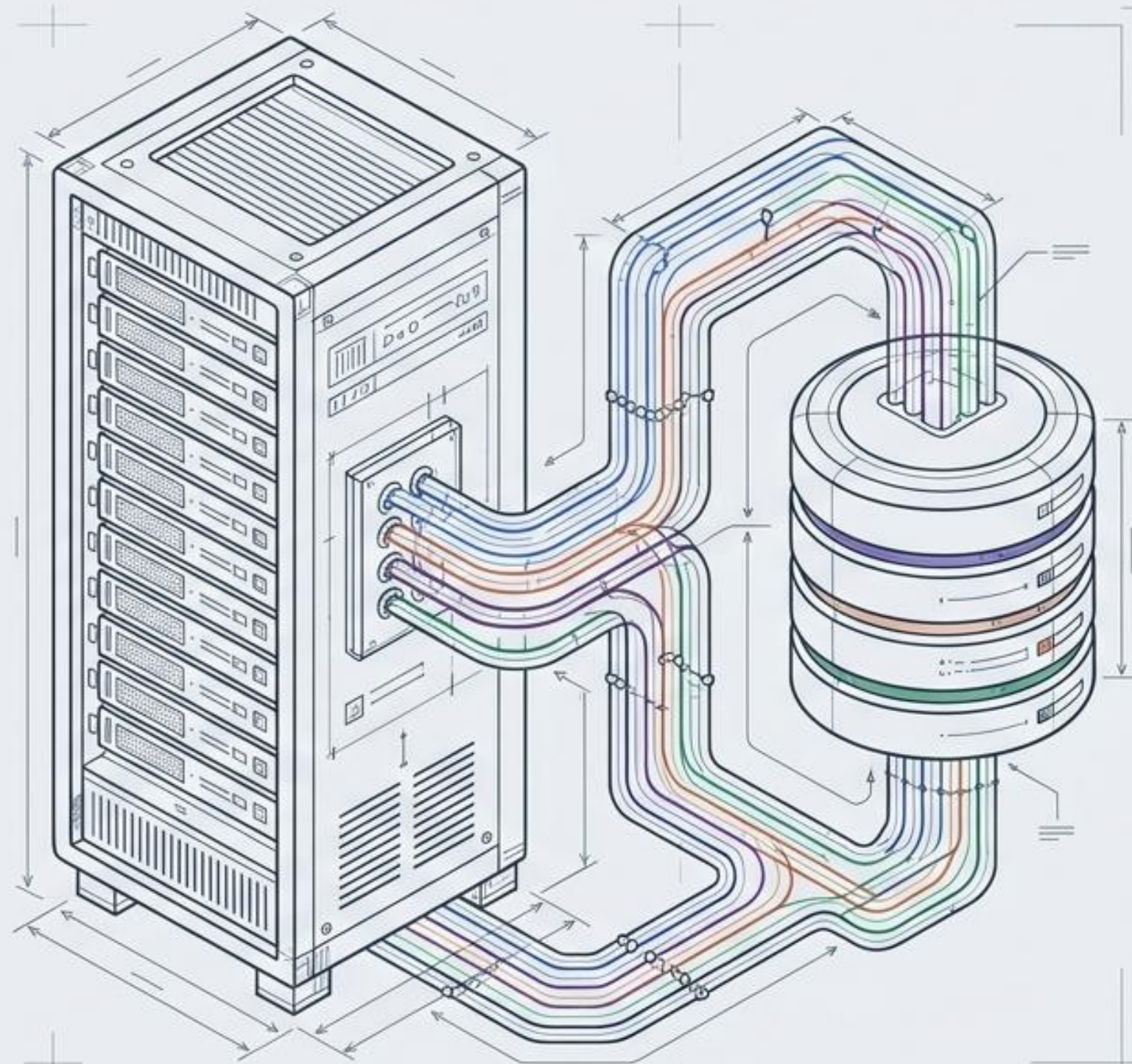
**구조적 우아함:** 복잡한 비즈니스 검색 로직을 단 하나의 쿼리 블록으로 우아하게 해결하는 MyBatis의 진가.



# Java Backend Masterclass: 4 Core Pillars

어노테이션, XML, MyBatis,  
그리고 MVC 패턴의 완벽한 해부

현대 자바 웹 애플리케이션을 구동하는 핵심 엔진 구축 가이드





# 자바 웹 어플리케이션을 구성하는 4개의 기둥



## 어노테이션 (Annotations)

코드를 지휘하는 스마트 포스트잇.  
컴파일러와 프레임워크에 직접  
명령을 내리는 메타데이터.



## XML

커스텀 데이터 상자.  
엄격한 규칙으로 데이터를  
구조화하고 의미를  
부여하는 설정의 근간.



## MyBatis

강력한 SQL 엔진.  
자바 객체와 데이터베이스를  
연결하고 동적 쿼리를 조립하는  
매퍼.

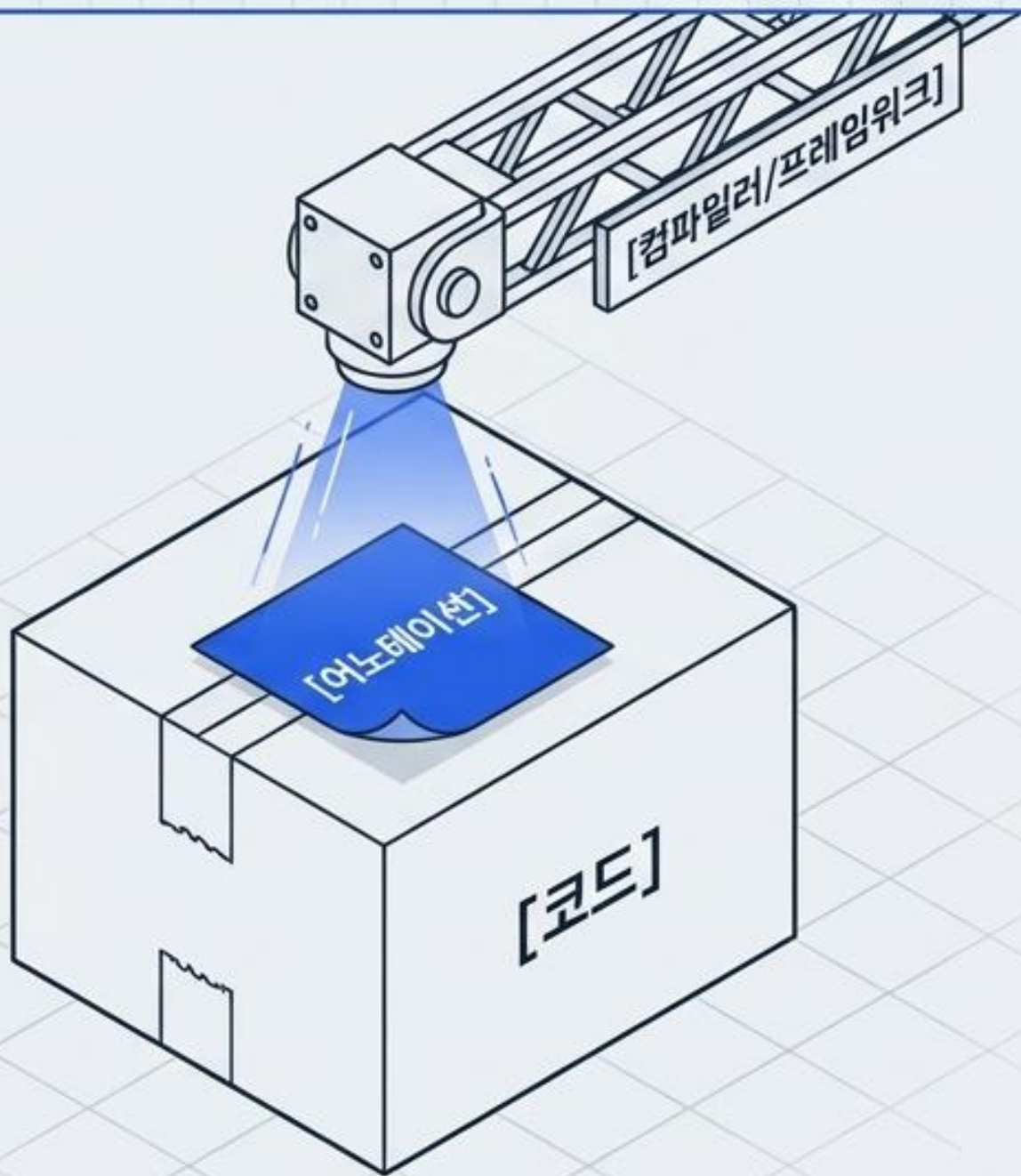


## JSP MVC 패턴

트래픽 컨트롤러.  
데이터(Model), 화면(View),  
제어(Controller)를 철저히  
분리하는 아키텍처.



# 어노테이션: 단순한 주석이 아닌 ‘스마트 포스트잇’



## 일반 주석 (//)

사람만 읽고 컴퓨터는 무시함.

## 어노테이션 (@)

컴파일러나 프레임워크가 읽고 특정 작업을 수행하도록 지시하는 특별한 마커.

## 존재 이유

과거의 분리된 복잡한 XML 설정 파일에서 벗어나, 코드 바로 위에 직관적으로 설정 정보를 적기 위함 (코드의 간결화 및 생산성 향상).



# 핵심 내장 어노테이션과 생성 규칙

## 3대 필수 내장 어노테이션

### @Override

부모 메서드 재정의 확인.  
컴파일러에게 '오타 감시'를 지시  
(ex. mymethod 오타 방지).

### @Deprecated

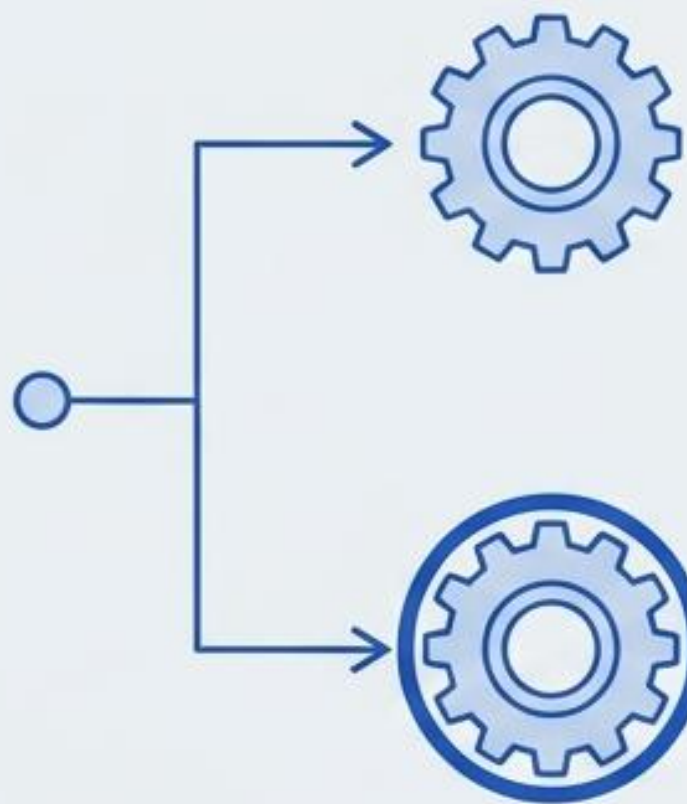
사용 권장 안 함. IDE에서 코드에  
취소선을 그어 곧 사라질 예정임을 경고.

### @SuppressWarnings

특정 경고창 끄기.  
(ex. 'unused' - 안 쓰는 변수 경고 무시).

## Recipe Blueprint

### 메타 어노테이션 (어노테이션을 만드는 어노테이션)



#### @Target (어디에 붙일까?)

- ElementType.METHOD (메서드),
- FIELD (변수),
- TYPE (클래스)

#### @Retention (언제까지 살아남을까?)

- 사용까지 관어잘소와 특정 정려도 가능  
(컴돈미이션 파일 유지)
- RetentionPolicy.SOURCE (컴파일 시 소멸)
- CLASS (클래스 파일 유지)
- RUNTIME (실행 중 유지 - 가장 많이 쓰임!)



# 어노테이션의 작동 원리 (주말 업무 차단기 예제)

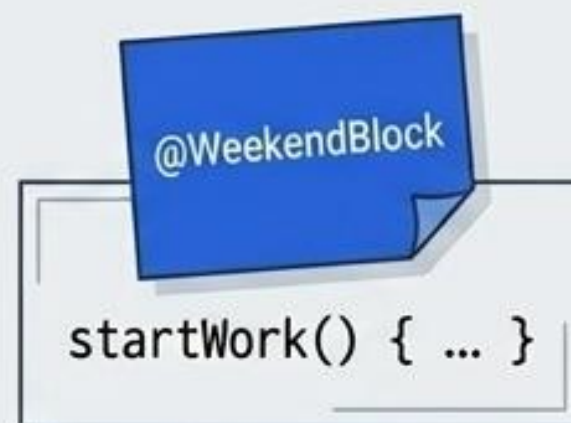
어노테이션은 스스로 작동하는 마법이 아닙니다. 프로그램을 통해 스티커를 검사해야 합니다.

## Step 1: 정의 (Define)

```
@Target(ElementType.METHOD)
@Retention(RetentionPolicy.RUNTIME)
public @interface WeekendBlock { ... }
```

## Step 2: 부착 (Attach)

@WeekendBlock을  
startWork() 메서드 위에  
부착.



## Step 3: 실행 및 검사 (Execute via Reflection)

- **로직:** 리플렉션(Reflection) 기술인 `method.isAnnotationPresent()`를 사용해 부착 여부 확인.
- **결과:** 주말(토/일)일 경우 스티커의 메시지를 출력하고 `return;`으로 메서드 실행 강제 종료.



# XML: 데이터를 담는 맞춤형 상자

## HTML vs XML 비교



**HTML:** 화면에 어떻게 보여줄지(디자인) 정의 (정해진 태그 사용).  
비유: 인쇄된 택배 상자.



**XML:** 데이터를 어떻게 구조화할지(의미) 정의 (태그 내 맘대로 생성).  
비유: 백지 상자에 직접 쓴 라벨.

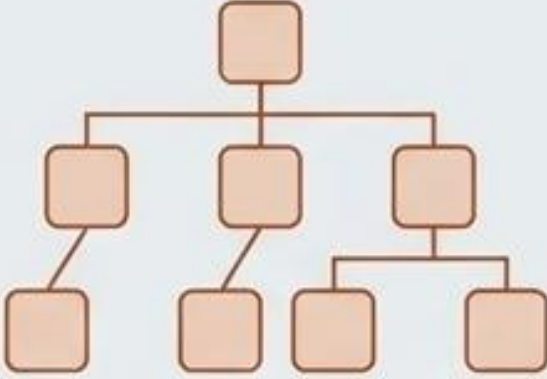
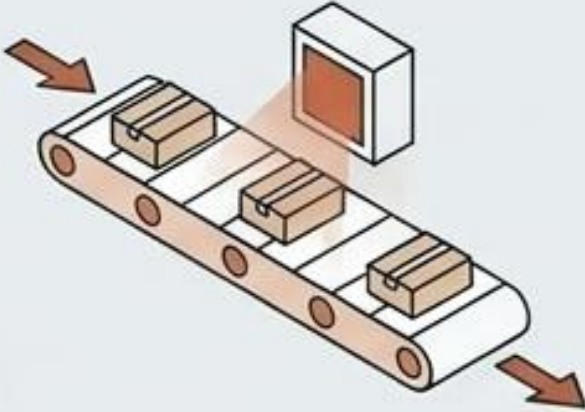
## XML 5대 절대 규칙 체크리스트

- ✓ 단 하나의 루트 엘리먼트: 전체를 감싸는 가장 큰 왕 상자는 하나뿐이어야 함.
- ✓ 반드시 태그 닫기: `<name>`은 반드시 `</name>`으로 끝날 것. (빈 태그는 `<br />`)
- ✓ 엄격한 대소문자 구분: `<Name>`과 `<name>`은 완전히 다른 태그.
- ⚠ ✓ 꼬임 없는 부모-자식 관계 (올바른 네스팅):  
○ `<a><b>값</b></a>` / ✗ `<a><b>값</a></b>`
- ✓ 속성값은 따옴표 필수: `<book id='b001'>` (따옴표 사용).



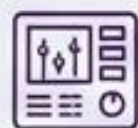
# XML 파싱(Parsing) 전략: DOM vs SAX

텍스트 덩어리인 XML을 자바 객체로 쪼개고 분석하는 두 가지 접근법.

| DOM 파싱 - Document Object Model                                                      | 작동 원리                                 | 장/단점                                                           | 추천 상황 Badge                                                                                                       |
|-------------------------------------------------------------------------------------|---------------------------------------|----------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
|   | XML 전체를 읽어서 메모리에 나무(Tree) 구조로 다 올려둬م. | 장점: 데이터 탐색, 수정, 추가가 매우 자유로움.<br>단점: 파일이 크면 메모리 소모가 극심함.        | <br>크기가 작은 XML, 데이터 가공 필요 시.   |
| SAX 파싱 - Simple API for XML                                                         | 작동 원리                                 | 장/단점                                                           | 추천 상황 Badge                                                                                                       |
|  | 위에서 아래로 한 줄씩 읽으면서 이벤트 발생 (단방향).       | 장점: 메모리를 거의 안 씀, 속도가 매우 빠름.<br>단점: 한 번 지나간 데이터 재조회 불가, 코딩 복잡함. | <br>대용량 XML 파일을 단순히 읽기만 할 때. |



# MyBatis의 이원화 아키텍처



## config.xml (컨트롤 룸)

**역할:** 전반적인 세팅  
(DB 연결, 플러그인, 매퍼 등록)

**!** 경고: <configuration> 내부 태그는 반드시 정해진 순서를 지켜야 함  
(순서가 틀리면 파싱 에러)

순서: ① — ② — ③ — ④ — ⑤  
properties → settings → typeAliases → environments → mappers



## mapper.xml (작업장)

**역할:** 실제 SQL 문 작성 및  
자바 객체(DTO/VO)와 데이터베이스  
테이블 매핑

<select>

<insert>

<update>

<delete>

<resultMap>



# 파라미터 바인딩: #{ } vs \${ } (보안 핵심)

## #{param}



- **작동 방식:** PreparedStatement 사용. 값에 자동으로 따옴표(')가 붙음.
- ✓ • **보안성:** SQL 인젝션 공격을 원천 방지함.
- **사용처:** 기본적으로 무조건 사용해야 하는 표준 방식.

## \${param}

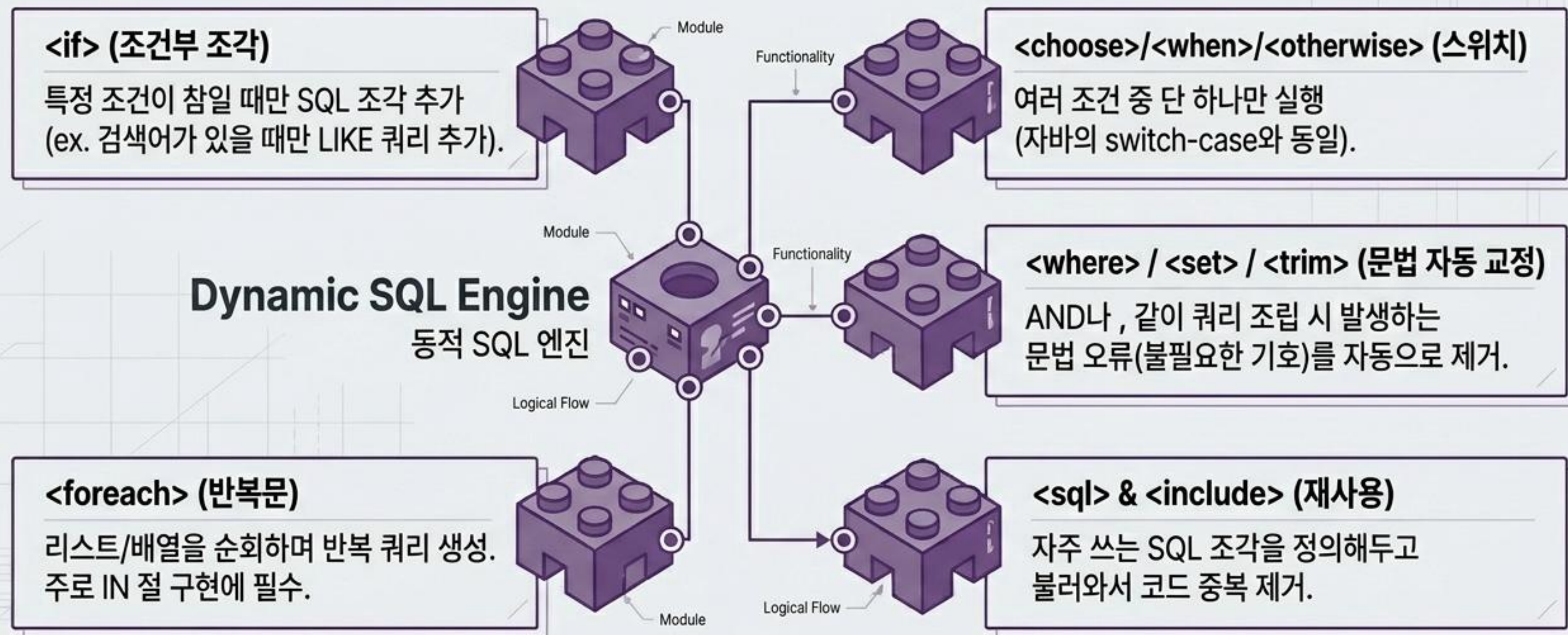


- **작동 방식:** Statement 사용. 값이 문자열 그대로 SQL문에 주입됨.
- ⚠ • **보안성:** SQL 인젝션 공격에 매우 취약함.
- **사용처:** 컬럼명, 테이블명, ORDER BY 절을 동적으로 바꿀 때만 극히 제한적으로 사용.



# 동적 쿼리 (Dynamic SQL): 스마트 쿼리 빌더

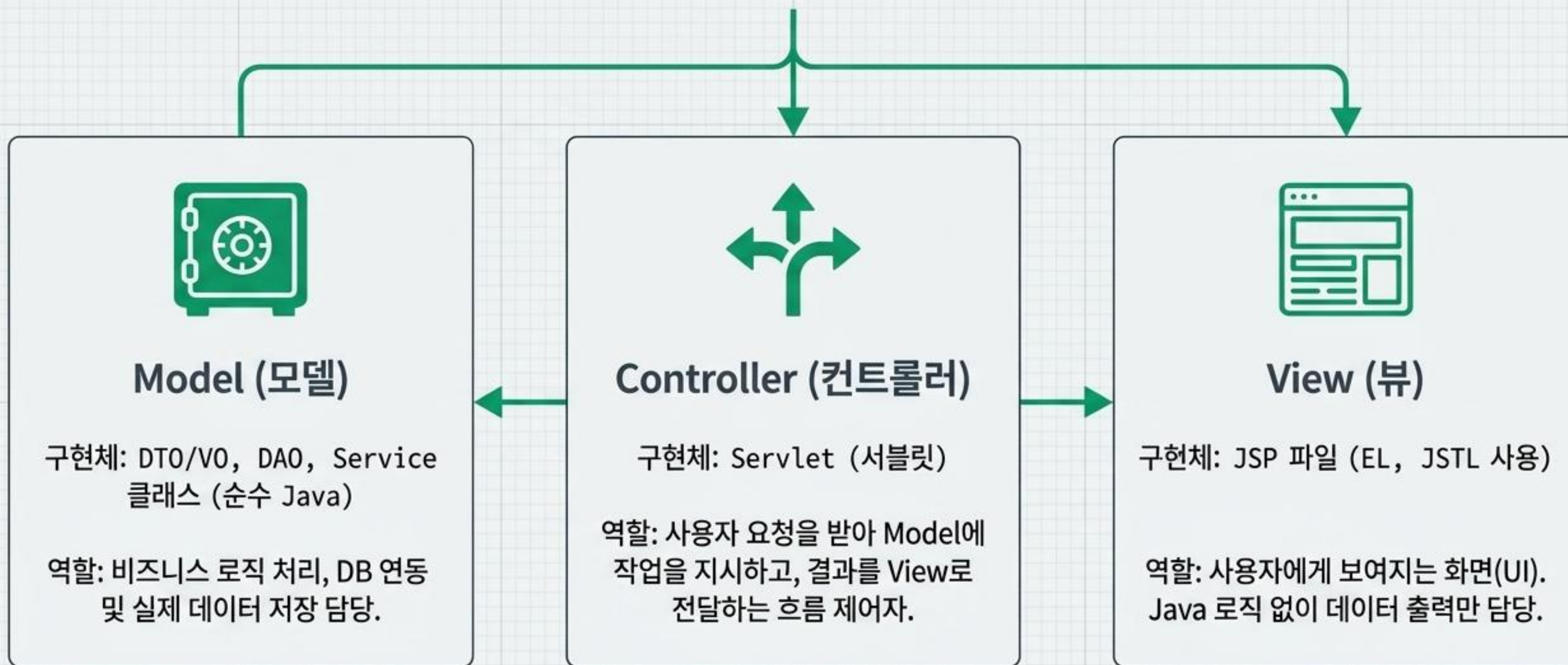
상황에 따라 SQL문이 스스로 형태를 바꾸는 MyBatis 최고의 무기.





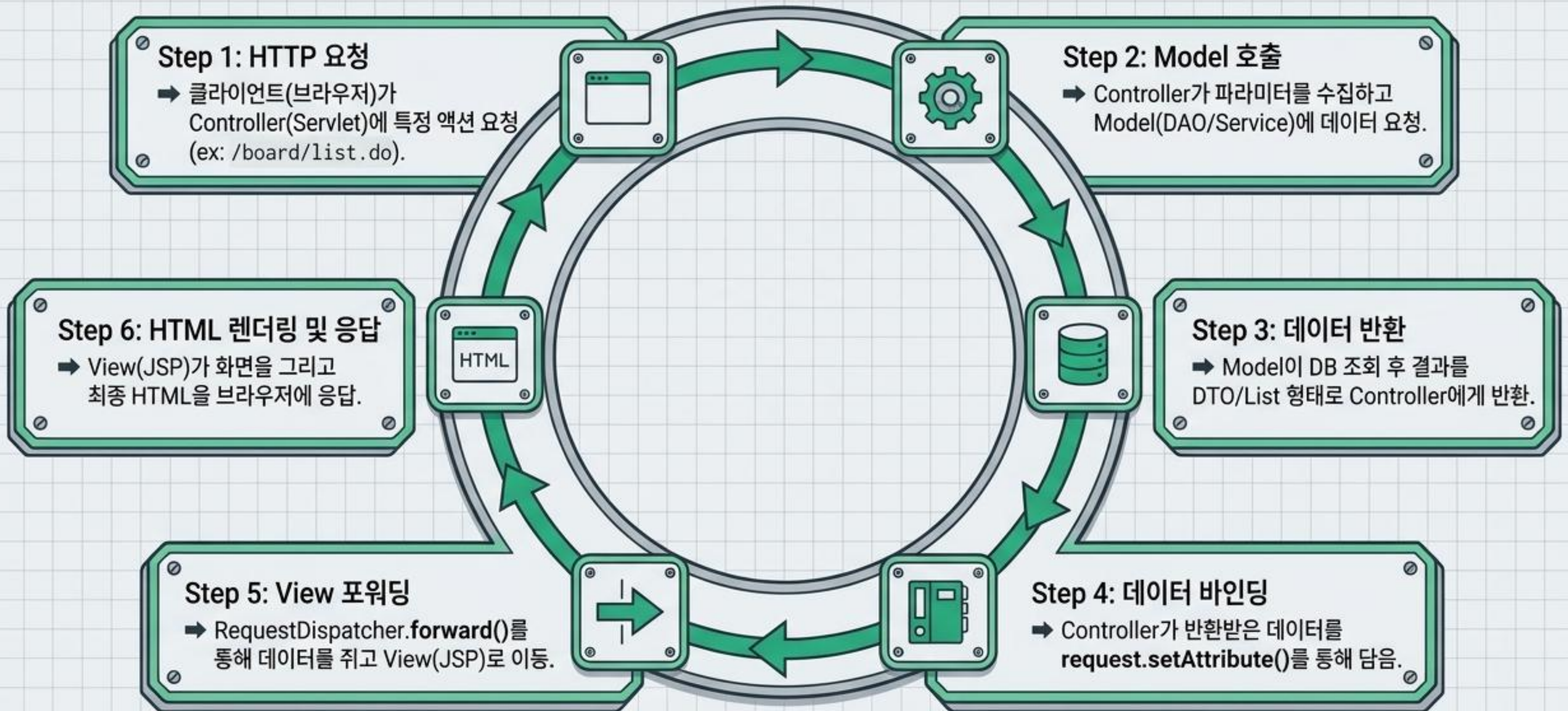
# JSP MVC (Model 2) 패턴 : 3요소의 철저한 분업

유지보수의 지옥이었던 Model 1(JSP 짬뽕 코드)을 벗어나, 화면과 비즈니스 로직을 분리한 현대적 구조.





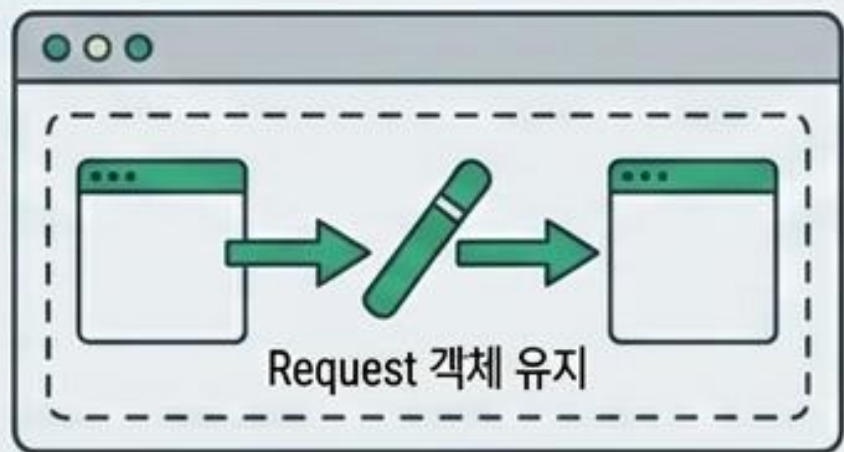
# MVC Request Lifecycle (요청 처리 6단계)





# 화면 이동의 핵심: Forward vs Redirect

## Forward - 포워딩



- ➔ 개념: 서버 내부에서 다른 페이지로 이동.
- ➔ 특징: URL 주소창이 변하지 않음.  
Request 객체가 유지됨.
- ➔ 용도: 주로 조회(SELECT) 수행 후 화면을 보여줄 때 사용.  
(`dispatcher.forward()`)

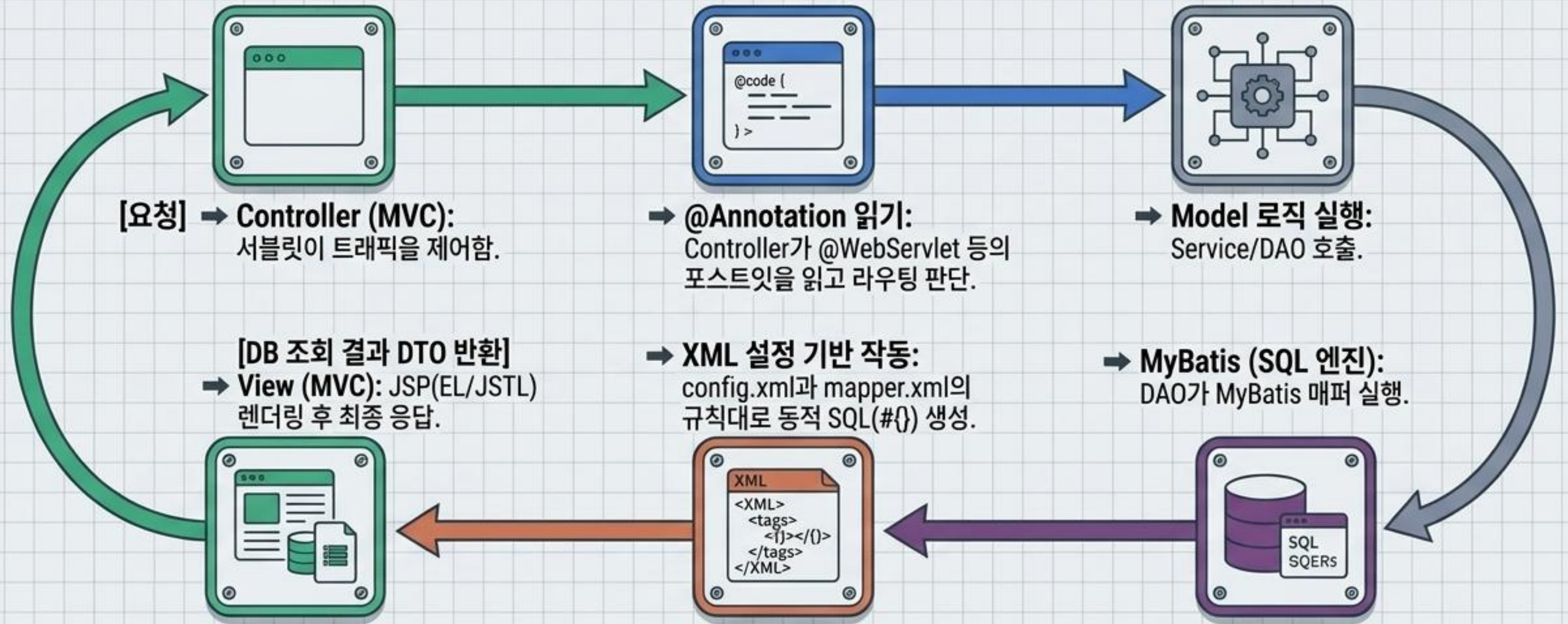
## Redirect - 리다이렉트



- ➔ 개념: 브라우저에게 "이 주소로 다시 접속해!"라고 명령.
- ➔ 특징: URL 주소창이 새 주소로 변함. 완전히 새로운 요청(Request) 생성.
- ➔ 용도: 등록/수정/삭제(CUD) 후 새로고침 시 중복 처리 방지를 위해 사용. (`response.sendRedirect()`)



# 아키텍처 합성: 자바 웹 애플리케이션 완전체



이 4가지 기둥은 독립적인 기술이 아니라, 하나의 거대한 톱니바퀴처럼 맞물려 작동하는 단일 시스템입니다.



# 핵심 요약 및 다음 단계 (Key Takeaways)

## 어노테이션 (Annotations)

XML 지옥에서 벗어나게 해준 직관적인 코드 위 메타데이터 (스마트 포스트잇).

[METADATA]

## XML

엄격한 5대 규칙과 파싱(DOM/SAX)을 통해 시스템을 단단하게 묶는 구조화된 설정 상자.

[STRUCTURE]

## MyBatis

보안("#{})과 동적 쿼리로 무장하여 자바와 DB 사이의 간극을 메우는 강력한 SQL 엔진.

[SQL\_ENGINE]

## MVC 패턴

화면(JSP)과 로직(Java)을 분리하여 유지보수성을 극대화한 웹 애플리케이션의 뼈대.

[ARCHITECTURE]

**단점의 극복:** 기능이 커질 수록 서블릿 파일이 무수히 늘어나는 MVC의 단점은, 모든 요청을 하나로 받는 Front Controller 패턴을 거쳐 현대의 Spring Framework (DispatcherServlet)로 진화하게 됩니다.